Machine Learning Based Risk Stratification for Lung Cancer Survival: Towards Kaplan-Meier Plots of Uniform Curve Separation

Ethan Lim

Supervisor: Hongkai Wen
Year of Study: 3rd Year

March 2026

Acknowledgements

I would like to thank my primary supervisor Hongkai Wen, and second supervisors Eric Lim and Minghao Xu for guidance throughout the project, as well as Kirsty Hassall for suggesting the idea of implementing a scoring function to evaluate KM plots.

Contents

Acknowledgements **1**

Abstract **4**

1 Introduction **5**

2 Research **5**
2.1 TNM Cancer Staging 5
2.2 Prognostic Models for Predicting Patient Outcome 6
2.3 Machine Learning Models for Predicting Patient Outcome 6

3 Objectives **6**
3.1 Producing KM Plot of Dataset 6

4 Exploratory Data Analysis **8**
4.1 Data Cleaning 8
4.2 Correlation Matrix 10
4.3 Boxplots for Covariates 12

5 Defining the KM Score Function **13**

6 Random Survival Forest **15**
6.1 Implementing a Random Survival Forest Model 15
6.2 Hyperparameter Tuning 16

7 DeepHit **24**
7.0.1 DeepHit Hyperparameter Tuning 26

8 DeepHit Evaluation **28**
8.0.1 Analysing Feature Importance of The DeepHit Model . . 31

9 XGBoost **38**
9.1 Hyper-parameter Tuning 39
9.1.1 Evaluating the XGBoost Model 44

10 Using SHAP Values to Understand XGBoost Model **47**
10.1 Findings from SHAP plot 48

11 Clustering for Hidden Covariate Relationships **50**
11.1 Evaluating the Cluster 53

12 Wald Test to Evaluate New Features **59**

13 Impact of Feature Engineered Covariates on the XGBoost Model **62**

14 Evaluation **63**
14.1 Reflection 65

15 Further Extensions **65**

A Appendix: Additional Material **69**

Abstract

Tumour Node Metastasis (TNM) staging is the current clinical standard for classifying lung cancer patients into risk groups, however its Kaplan-Meier (KM) plots produce survival curves of unequal separation over time, limiting its discriminatory power and clinical utility. This report investigates the efficacy of machine learning models as classification mechanisms for a novel risk labelling system designed to produce KM plots whose survival curves maintain uniform separation over time. A KM scoring function was developed to quantitatively evaluate the quality of produced KM plots, penalising curve intersection, ordering violations, incomplete curves and deviation from uniform separation. Three models were implemented and evaluated using this scoring function as the optimisation objective within a Bayesian optimisation framework: a Random Survival Forest (RSF), a DeepHit deep learning model, and an XGBoost model with a survival Cox objective. The dataset used was extracted from the Surveillance, Epidemiology and End Results (SEER) program, comprising over one million lung cancer patient records. Exploratory data analysis, hierarchical clustering and SHAP value analysis were conducted to identify influential covariates and hidden feature interactions, leading to the development of feature engineered covariates that were validated using the Wald test prior to inclusion in model training and optimisation. Results demonstrated that the RSF model achieved the lowest KM score of 0.203, outperforming both DeepHit, which failed to produce valid KM plots due to survival curve intersections, and XGBoost, which achieved a best KM score of 0.307 following feature engineering. Whilst no model produced KM plots of perfectly uniform separation, the RSF model produced survival curves covering a greater number of risk profiles and maintaining survival probabilities above 0.4 at the ten year time point, representing a visual improvement over the four stage TNM system.

Key words: Tumour Node Metastasis (TNM), Kaplan Meier (KM), Random Survival Forest, DeepHit, XGBoost, KM score, SEER, Uniform separation.

1 Introduction

Tumour Node Metastasis (TNM) staging has been used by doctors for decades to:

- Understand how serious a patient's cancer is [21]
- Understand their chances of survival [21]
- Plan the best treatment for their patient [21]
- Identify clinical trials that may be the best treatment option for them [21].

However, the issue with the current TNM system is that its Kaplan-Meier plot (KM plot) produces survival curves that are unequal in separation over time. This is because TNM is a categorical anatomical system and not a risk score [19]. Curves of unequal separation imply that the difference in risk between stages varies. In addition, for each stage to maintain constant discriminatory power, the separation between curves should remain consistent over time. Furthermore, a more informative system should have more stages to allow for finer risk stratification, allowing for more tailored clinical decision-making [19].

Using machine learning as a classification mechanism may offer advantages over predefined rule-based algorithms by incorporating factors beyond TNM. Moreover, a machine-learning-based approach enables stages to be derived directly from the models risk estimates, allowing patients to be assigned to predefined risk brackets in a manner that ensures survival curves of uniform separation.

2 Research

2.1 TNM Cancer Staging

The T in the TNM system refers to the size and extent of the main (primary) tumour (where the cancer originated), The N refers to the number of nearby lymph nodes that have the cancer and the M refers to whether the cancer has metastasized, which means that the cancer has spread from the primary tumour to other parts of the body [21].

T1, T2, T3, T4 refer to the size and/or extent of the main tumour. The higher the number after T, the larger the tumour or the more it has grown into nearby tissues [21].

The issues with the TNM system are:

- Wide prognosis within the same stage, patients assigned to the same TNM stage can have very different survival outcomes, so stage does not tightly correspond to risk [19].
- Lack of biological and patient-level factors [5].

2.2 Prognostic Models for Predicting Patient Outcome

In recent years, *risk prognostic models* have become increasingly popular to aid clinical decision-making [9]. Examples of prognostic models used in clinical practice include the *Wells rule* for patients suspected of *pulmonary embolism*. For a specific set of input variables, given by a patient, a prognostic model provides an estimated probability that allows for risk stratification for individuals or groups. Prognostic models are usually derived using *multivariable regression techniques*. The advantage these models have over machine learning models is that there is no 'black box' which leads to issues of trust and accountability [22].

2.3 Machine Learning Models for Predicting Patient Outcome

Shallow machine learning models have already been used for clinical studies, such as logistic regression for predicting *hemodynamic instability* [3] and mortality [7], and decision trees for developing an *early warning score* [11].

One study [13] claims that although decision trees offer high interpretability, they heavily rely on task-specific hand-engineered features and do not learn complex patterns in the data.

Deep learning models are also becoming increasingly popular for outcome prediction tasks [13]. Both Long Short Term Memory (LSTM) networks [8] and Gated Recurrent Units (GRU) [6] were developed to predict clinical outcomes. Although deep learning techniques are typically characterised by strong performance, their decision making process lacks interpretability [13].

3 Objectives

The points below outline the objectives for this project.

- Investigate the efficacy of a shallow learning model as a classification method for a labelling system that leads to a KM plot whose survival curves are of uniform separation.
- Investigate the efficacy of a deep learning model as a classification method for a labelling system that leads to a KM plot whose survival curves are of uniform separation.

3.1 Producing KM Plot of Dataset

In order to demonstrate the flaws of the current TNM cancer staging system, a KM plot grouped by a patient's TNM cancer stage was produced using the SEER dataset, a public dataset containing over a million lung cancer records.

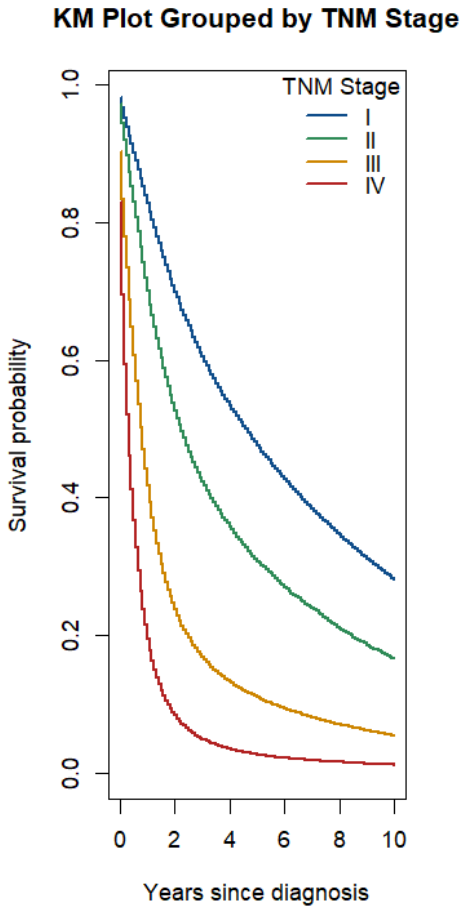


Figure 1: KM plot of patients grouped by TNM stage.

Figure 1 demonstrates that the survival curves corresponding to different TNM stages are not evenly separated. Moreover, the degree of separation between the curves varies over time, indicating that the discriminatory power of TNM staging is not consistent across the follow-up period.

4 Exploratory Data Analysis

The dataset used in the project was extracted from the Surveillance, Epidemiology and End Results (SEER) Program, which is a national cancer registry program run by the National Cancer Institute (NCI) in the United States. As the dataset (named 'SEER Research Data, 17 Registries, Nov 2024 Sub') had over 17 million rows, the SEER Stat Application, a statistical software program provided by the NCI, needed to be used in order to filter for patients with lung cancer.

4.1 Data Cleaning

Covariates in the dataset representing numeric values were originally represented as bins. In addition, the columns types of these covariates were originally categorical. In order to allow the machine learning models to interpret order within these covariates, these bins were replaced by their midpoints, before the covariate was converted into a numeric object. Values encoded numerically to represent 'Unknown' were converted to pd.NA so that they are represented in the dataset as missing values. By removing all rows containing a missing value from the dataset, the remaining dataset is empty, indicating that the dataset has high noise. The table in the next page lists all the covariates within the cleaned dataset as well as their corresponding number of missing values. The filtered dataset contained over 1 million rows, and was downloaded using the SEER Stat application.

Feature	NA Count
Age recode with < 1 year olds and 90+	27
Sex	0
Year of diagnosis	0
Race recode (W, B, AI, API)	2201
Origin recode NHIA (Hispanic, Non-Hisp)	0
Race and origin recode (NHW, NHB, NHAIAN, NHAPI, Hispanic)	1519
Marital status at diagnosis	47107
Median household income inflation adj to 2023	17437
Rural-Urban Continuum Code	1637
Survival months	0
AYA site recode 2020 Revision	1
ICCC site recode 3rd edition/IARC 2017	0
Primary Site - labeled	0
Behavior recode for analysis	0
Grade Recode (thru 2017)	688017
Derived Summary Grade 2018 (2018+)	845403
Grade Clinical (2018+)	845403
Grade Pathological (2018+)	845403
Laterality	0
Diagnostic Confirmation	18307
Histology recode - broad groupings	0
ICCC site recode extended 3rd edition/IARC 2017	0
Schema ID (2018+)	845403
Site recode - rare tumors	138
Tumor Size Over Time Recode (1988+)	280268
Tumor Size Summary (2016+)	805920
Regional nodes examined (1988+)	138144
Regional nodes positive (1988+)	841217
SEER Combined Mets at DX-bone (2010+)	492173
SEER Combined Mets at DX-brain (2010+)	492983
SEER Combined Mets at DX-liver (2010+)	492662
SEER Combined Mets at DX-lung (2010+)	494838
Mets at DX-Distant LN (2016+)	760084
Mets at DX-Other (2016+)	759358
Vital status recode (study cutoff used)	0
First malignant primary indicator	0
Total number of in situ/malignant tumors for patient	0

Table 1: Table to show missing values by covariate.

Within table 1, over 70% of patients do not have data for covariates such as 'Grade Clinical (thru 2017)' and 'Derived Summary Grade 2018 (2018+)'. High levels of missing data for particular covariates arise from inconsistent data collection across institutions [4]. Covariates with the suffix (2018+) or (2016+) have extremely high missing value counts likely because patients diagnosed before those years have no recorded value for these fields.

4.2 Correlation Matrix

A correlation plot for numerical covariates only was implemented to identify and remove redundant covariates. Any pair of covariates with an absolute covariance above 0.8 (threshold value) were to be investigated for removal. Removing redundant covariates will aid model interpretability and general model processing speed.

A correlation matrix was used rather than a covariance matrix, since correlation values are scale-independent and bounded to [-1,1], making cross-covariate comparisons more interpretable.

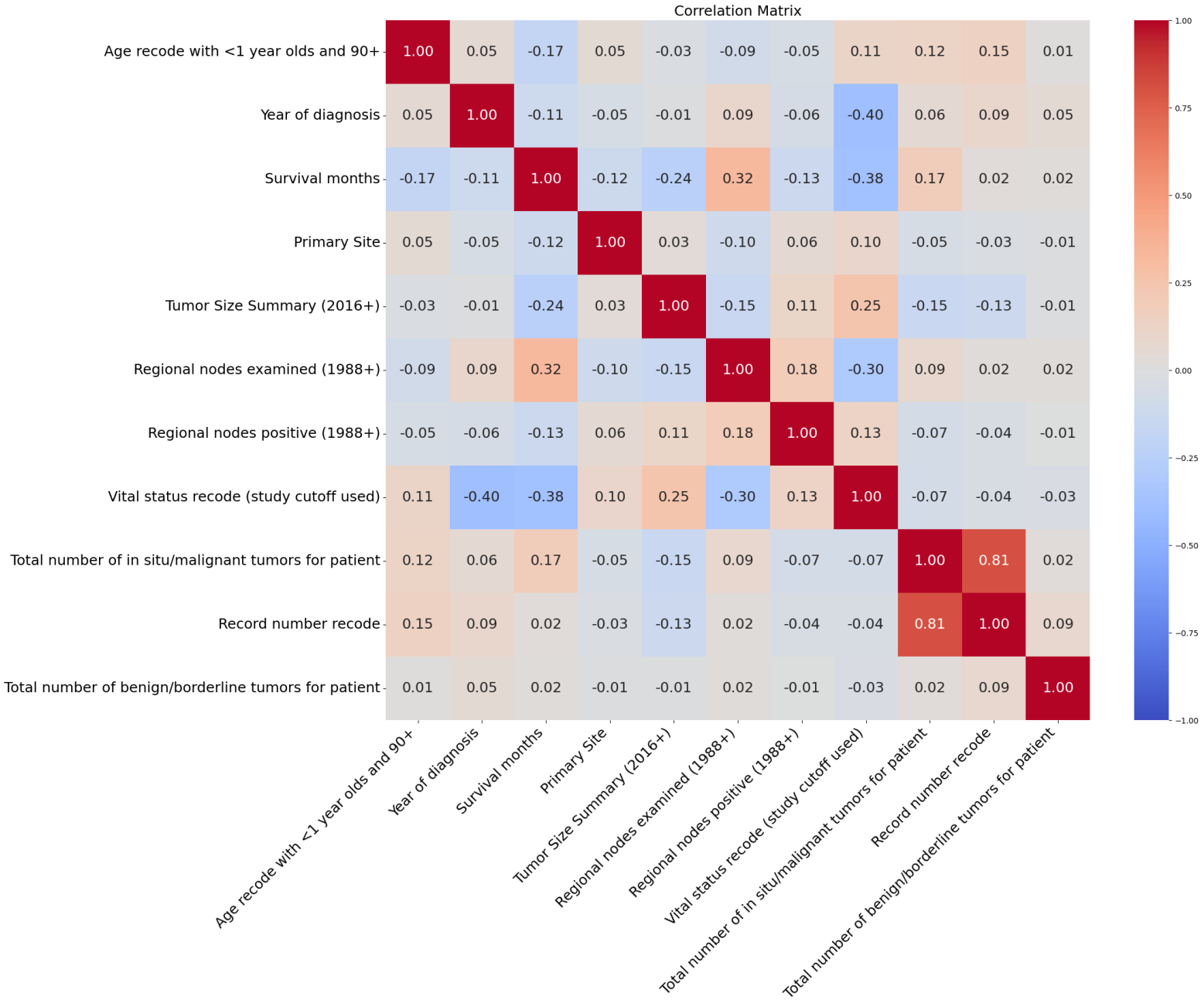


Figure 2: Correlation matrix using all the covariates within the dataset

The only pair of covariates with a an absolute correlation above 0.8 are the covariates 'Total number of in situ/malignant tumors for patient' and 'Record

number recode'. This is because 'record number recode' sequentially numbers a person's tumors, so a person with a higher number of tumors will be given a higher record number. Since 'Record number recode' stores the same information as 'Total number of in situ/malignant tumors for patient', the 'Record number recode' covariate was removed.

4.3 Boxplots for Covariates

Boxplots for numerical covariates were investigated to better understand distribution of numerical covariates.

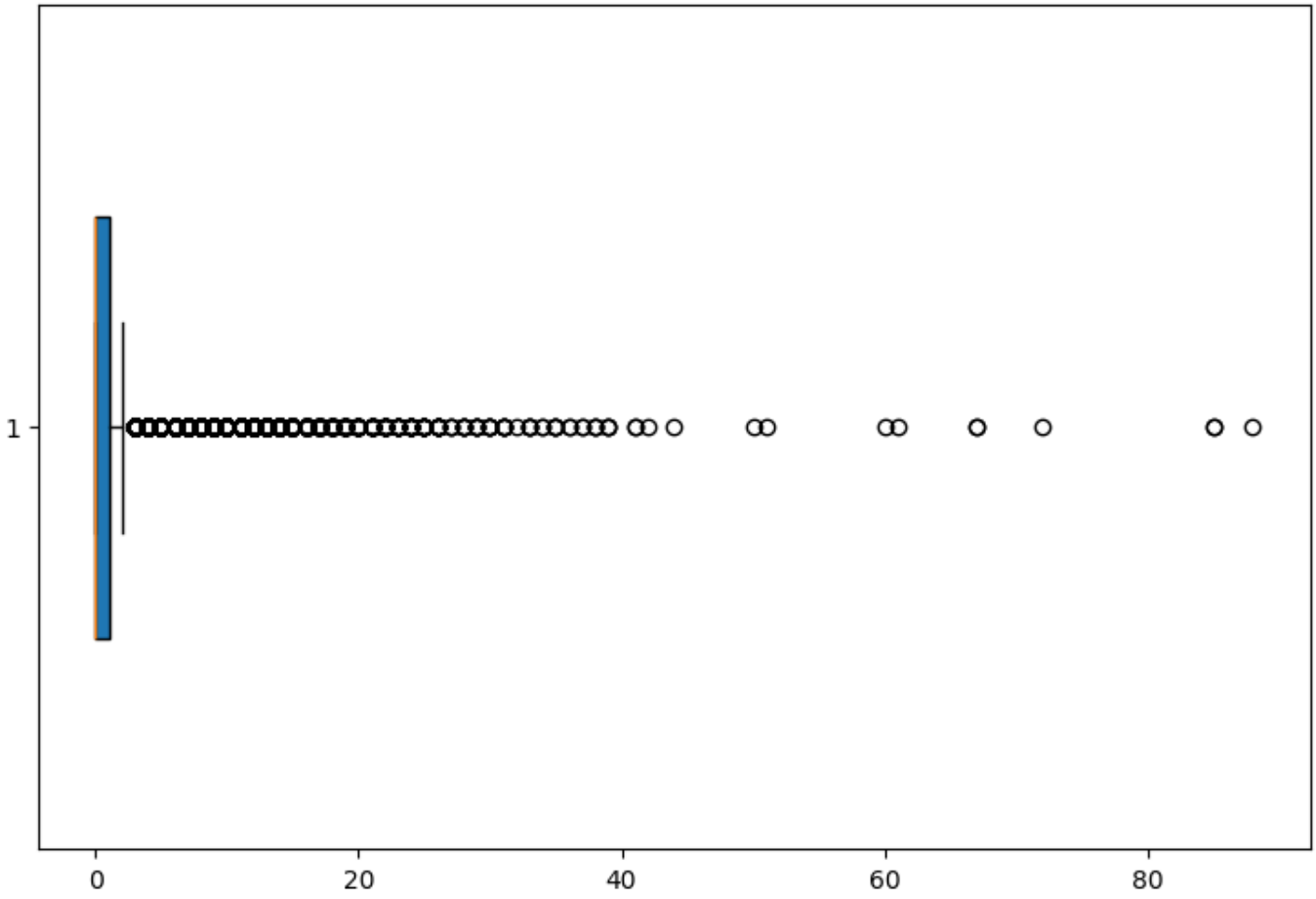


Figure 3: Boxplot to show distribution of regional node sizes amongst patients within the dataset.

The narrow width of the box and whiskers imply there is one dominant

cluster of patients (number of data points for regional node number and tumor size differ due to missing values).

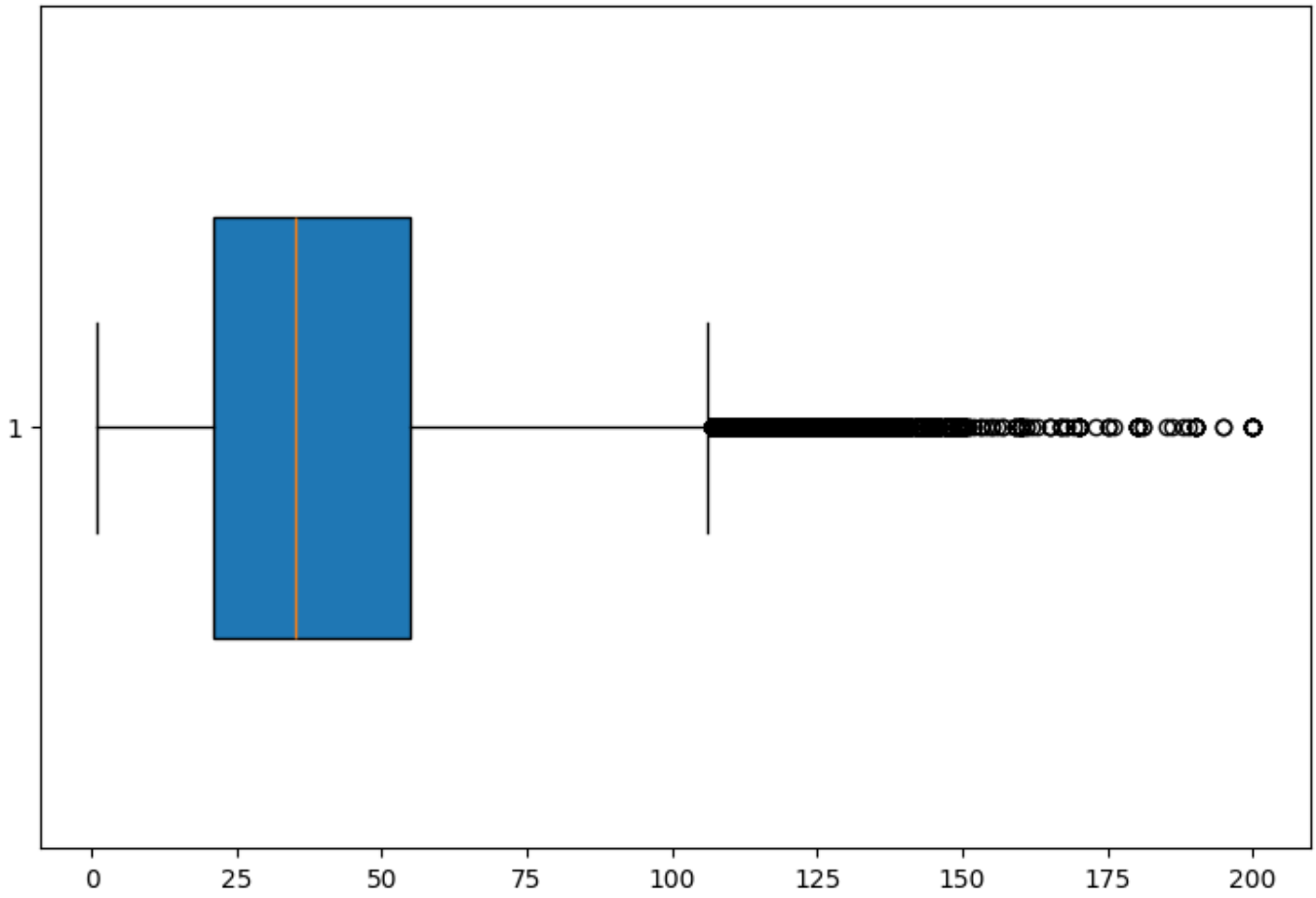


Figure 4: Boxplot to show distribution of tumor size over time amongst patients within the dataset.

595,993 patients are below the value for Q_3 and 205264 patients are above this value. The maximum whisker may imply that there is a second cluster of patients causing the interquartile range to increase.

5 Defining the KM Score Function

A valid KM plot whose survival curves follow equal separation must have the following properties:

- (**Completion**) All curves extend throughout the entire length of the x axis, which represents the time index for survival duration.
- (**Parallelism**) The separation between survival curves remains constant over time.
- (**Uniform Separation**) Curves are equally spaced with each curve taking up an equal portion of the probability range (from 0 to 1).
- (**Order**) The position of survival curves should be descending as the risk label increases.
- (**No cross over**) At no point should curves intersect with each other,

The table below shows main feature implemented within the scoring function that satisfy the requirements above (see Listing 1 in the Appendix for the implementation). Therefore, a scoring function that ensures these requirements are met must have all of the following:

Trait	**Justification**
A penalty for curves that are incomplete up to the end of the x axis.	This will address the **completion** objective.
A method to calculate the absolute difference between the separation of consecutive curves and the optimal separation value throughout all time points.	This will satisfy the **parallelism** and **uniform separation** objectives.
A method to detect and intersection between curves and return the worst score.	This will satisfy the **no cross over requirement**.
A method to return the worst score if the order of survival curves is not preserved.	This will satisfy the **No cross over** requirement.

Table 2: Table to show the traits the scoring function needs to satisfy all requirements.

In order to satisfy the requirements above, for every curve plotted within the KM plot, the lifelines kmf.survival_function_ attribute (a dataframe containing a column for the timeline and another column for the corresponding survival probabilities) was implemented.

A new data frame was created with a column representing a timeline. For each curve plotted, the probabilities for the curve were added as a column to the new dataframe, these probabilities were correctly indexed through matching the timeline column in the new dataframe with the timeline column within the kmf.survival_function attribute for that curve.

Pairs of columns representing consecutive survival curves were selected for row-wise iteration, where the difference between their probabilities for the time

index were calculated, and compared with the optimal separation via the following formula:

$$difference = |optimal_separation - |survival_probability_2 - survival_probability_1||$$

The $difference$ variable represents the absolute distance between the desired separation between the absolute difference of the probabilities belonging to a pair of survival curves. The differences are summed together to provide an overall score, with lower scores representing a better KM plot.

For any missing probability found when the differences are calculated, a missing penalty of value of 1 is used to represent the difference. The penalty of size 1 was chosen so that the optimiser prioritises a hyperparameter configuration that leads to curves that have a value for each time point (to avoid incomplete survival curves). This is because the value 1 is higher than the worst case difference between the separation of the survival curves and the optimal separation, as the separation between curves cannot exceed 1.

If the number of columns produced in the KM plot does not match the intended number of curves, a penalty score of 1000 is returned, in order to prevent such a case from happening. This case was implemented since pair differences and missing values are only calculated from the columns within the new dataframe, hence without a checking that the number of curves represented in the dataframe match the intended number of curves, good KM scores are generated even with the resulting KM plot having less survival curves than intended.

The score returned has been defined by the author to by the Kaplan Meier (KM) score. The desired and target KM plot for this project should have a KM score of 0.

6 Random Survival Forest

6.1 Implementing a Random Survival Forest Model

To evaluate the effectiveness of a shallow model for classification, a Random Survival Forest (RSF) model was implemented. Among the shallow learning models considered, RSF was selected because it is specifically designed to model time-to-event outcomes [10], allowing both censored and event observations to be incorporated for survival estimation. In addition, RSF can capture nonlinear relationships [10], unlike many other shallow learning models.

The RSF model outputs log hazard ratios, which represents a patients risk relative to other patients within the dataset. These log hazard ratios can be converted into survival probabilities using the Breslow estimator [2] and the following formula:

$$S(t \mid \mathbf{x}) = \exp\left(-H_0(t) \cdot e^{\boldsymbol{\beta}^\top \mathbf{x}}\right)$$

The original classification method involved converting each log hazard ratio into survival probabilities. Each patient would be assigned a risk label based on the survival probability bracket they were assigned to. However, the results led to missing curves since no patients were predicted a probability that fell into the probability bracket assigned to a risk label (note that each curve corresponds to a risk label). Since the survival probabilities determined by the model are not the survival probabilities generated by the Cox Proportional Hazards (CoxPH) model used to create the survival curves for the KM plot, the survival probabilities for the same patient using the RSF model and the CoxPH model will be different. Assigning patients based on their relative risk compared to other patients (represented by the log hazard ratios output by the RSF model) yielded risk labels that had more similarity in patient size. The results are reasonable since the relative difference in patient survival probability is directly mapped by their difference in their log hazard ratio.

Therefore the labelling system that led to a lower KM score was as follows:

- Train the model on the patient training dataset.
- Have the RSF model output predictions for the same training dataset.
- From the outputted log hazard ratio predictions, create bins using the lowest and highest ratio outputted log hazard ratios, such that the number of bins is equal to the intended number of survival curves, with the first bin starting from negative infinity and ending at the minimum outputted log hazard ratio, and the last bin starting from the maximum outputted log hazard ratio and ending at positive infinity.
- Have the model predict the log hazard ratio on the patient **test** dataset.
- Assign a risk label to each bin
- Assign each patient in the test dataset the corresponding risk label based on the bin that their model outputted log hazard ratio falls into.
- Send the patients survival durations, event statuses and corresponding risk labels to the KM plotter function.
- The KM function will use the CoxPH model to create the KM plot as well as return the KM score corresponding to the plot.

The training and testing dataset was split 80/20, as this split tends to provide better results than other splits in most cases.

6.2 Hyperparameter Tuning

Because Bayesian optimisation can find equal or better parameter configurations than Grid search or Random search [27, 26], Bayesian optimisation was implemented for hyperparameter tuning. Another advantage of Bayesian optimisation over Grid search and Random Search is that its search space is continuous instead of discrete (Grid search and Random search involve inputting a list

of discrete values to be considered per hyperparameter). Skopts gp_minimize() method was used to implement Bayesian optimisation. The evaluate_model argument passed to gp_minimize() was a function made from scratch, designed to train the model and use its predictions to create and assign risk labels to patients in the validation set, before passing the risk labels along with the validation set to the KM plotter to retrieve and return the KM score. The returned KM score is used to evaluate the hyperparameter configuration chosen by the optimiser.

In order to attain the best hyperparameter configuration without data leakage from the test data. The training data was split into training and cross validation folds and used to test and train the models with the hyperparameter configurations set by the Bayesian optimiser model. The optimised hyperparameter values were then passed to a new XGBoost model, before it is trained on the entire training dataset and its predictions made on the testing dataset.

Sklearn's RepeatedStratifiedKFold() method was implemented to split the training data into training and testing splits.

The range of the search space used during Bayesian optimisation for the Random Survival Forest models hyperparameter values are provided below.

Parameter	**Value**	**Description**	**Justification**
n_estimators	[10,100]	Number of trees in the random forest ensemble	The upper bound of the search space was set to 100 since higher values would cause the Jupyter Kernel to undergo a memory overload. The lower bound has been set to 10 as too few trees can yield unstable (high-variance) estimates, whereas performance typically stabilises as more trees are added, which is necessary for Bayesian optimisation since the next chosen configuration is determined by the outputted score.
max_depth	None	Limits the maximum number of splits from the root to any leaf node in each tree	Set to none in order to leave the depth unrestricted.

min_samples_split	[2,50]	Minimum sample size that reaches a node before it can attempt to create a split	A minimum split should have enough samples to create two valid leaves, meaning the lower bound should be at least 2. The upper bound was set to 50, this is because larger values tend to produce very shallow trees, leading to underfitting. There is no rule of thumb to determine the upperbound, however if the optimised configuration is near the upperbound, the next upperbound will be reconfigured to a higher value.
min_samples_leaf	[5,30]	The minimum number of training samples required to form a leaf node in each tree	This range was chosen to span from weak to strong regularisation, small values allow finer partitioning, whilst larger values enforce larger leaves and smoother predictions (predictions that are not overly sensitive to changes in covariate values)[23, 15].
min_weight_fraction_leaf	0.0	Sets the minimum weighted fraction of the total training samples required to be at a leaf node.	This was set to the default value of 0 since a search space for min_samples_leaf has already been provided.
max_features	[1,35]	Controls how many features each tree considers when looking for the best split at each node.	The maximum value has been set to 35 (number of columns in the dataset) to provide the full range of possible values.
max_leaf_nodes	[50,1000]	The maximum number of leaf nodes a single tree is allowed to have.	This range allows for the model to have trees of varying complexity.
bootstrap	True	This hyperparameter determines whether bootstrapping should be used	This hyperparameter has been set to true to reduce the variance of model predictions.
oob_score	True	This enables the calculation of internal out-of-bag score when the bootstrap hyperparameter is set to True. The score is then output.	The parameter is set to true as a way to provide signal on whether the model is overfitting.

max_samples	[10000, len(X_train) * 0.5]	Maximum number of samples drawn per tree	The lower bound of 10000 was chosen to avoid overly small subsamples per tree, which can increase the variance/instability of the fitted ensemble and degrade predictive performance [24, 15]. The upper bound was chosen to be half the number of observations in the dataset to keep bootstrapped samples meaningfully smaller than the full dataset during cross validation [24].

Table 3: RandomSurvivalForest Model Parameters

The evaluation function used within the Bayesian optimiser was implemented from scratch to optimise the KM score. The function contains the model initialisation process as well as the fitting process. In addition, it is where the risk labels are generated and the KM score is produced.

A two fold validation test was created to compare two optimised hyperparameter configurations together. Only two folds were chosen in order to minimise compute time whilst allowing comparison for how different folds within the dataset affect the KM score and the hyperparameter configuration.

The skopt.gp_minimize() method was used to implement the Bayesian optimiser [25]. The value of n_calls (number of evaluations) was set to 50 as a general rule of thumb to balance exploration of the hyperparameter search space with compute times.

Although both Bayesian optimised hyperparameter configurations were different to each other, both configurations when applied to a new RSF model, trained using the training data and tested using the testing data, attatined relatively similar scores of 0.203 (3sf) and 0.205 (3sf) on the unseen testing data. For the testing data in which the optimised configurations were trained on, the first configuration (config 1) attained a score of 0.192 (3sf) and the second configuration attained a score of 0.190 (3sf). The table below show the optimised hyperparameter values (whose models were trained and tested on separate halves of the overall training dataset) and their interpretations:

[H] **Hyperparameter**	**Config 1**	**Config 2**	**Interpretation**
n_estimators	51	12	For both configurations, the optimal number of estimators were not close to the upper bound (100) of the search space. This implies that the model does not need to make full use of the tree number in order to capture all the possible relationships it can identify. Potentially the model has reached its limit on the feature interactions and pattern that could be captured, and a higher number of estimators beyond 51 would lead to overfitting.
min_samples_split	18	6	Config 1s value of 18 suggests the dataset is noisier or smaller, needing more evidence of a general pattern before making a split, whilst config 2s value of 6 suggest the dataset is cleaner, allowing splits for more specific patterns.
min_samples_leaf	9	8	Since the values are similar, both configurations agree that the leaf nodes need a reasonable number of samples, implying that the dataset has enough noise that very small leaves would be unreliable.

max_features	25	35	The second configuration considers all the covariates for each split, suggesting that each feature is informative. The first configuration of 25 is of moderate size, still implying that most features are informative, but reduced to avoid over fitting during training.
max_leaf_nodes	64	54	Both values are of moderate size, with the upper bound of the search space set to 1000, this value implies that neither datasets require complex trees.
max_samples	118300	38970	The first configuration draws nearly three times more samples per tree, suggesting that the testing fold used for config 2 had higher variance.

Table 4: Optimal Random Survival Forest Hyperparameters and Interpretations

The first configuration shows more trees than the second configuration, but each tree is more simple, capturing more general patterns for each one. The second configuration has trees that focus more on capturing more complex patterns, given that each split considers all the covariates when deciding on a splitting rule.

Each configuration corresponds to a specific half of the dataset, used during optimisation. Constrasting configurations imply each half of the dataset had different patterns, it does not imply noise since the perfomance of the configurations on unseen test data yielded similar KM scores (0.203 and 0.205).

Outlined below are the KM plots corresponding to the risk labels generated by the model with the optimal configured hyperparameter values:

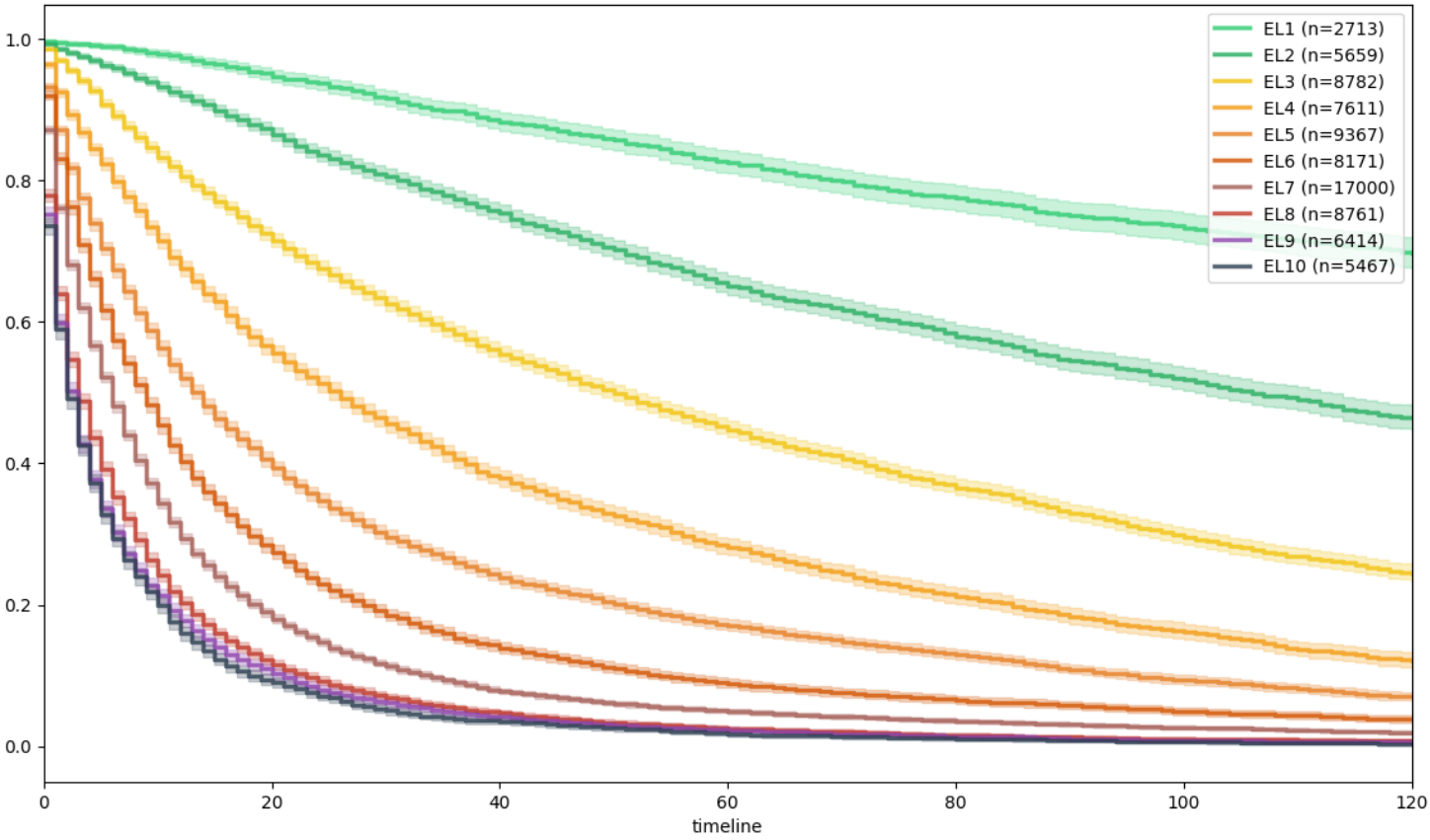


Figure 5: KM plot using the first optimal configuration on the unseen test data

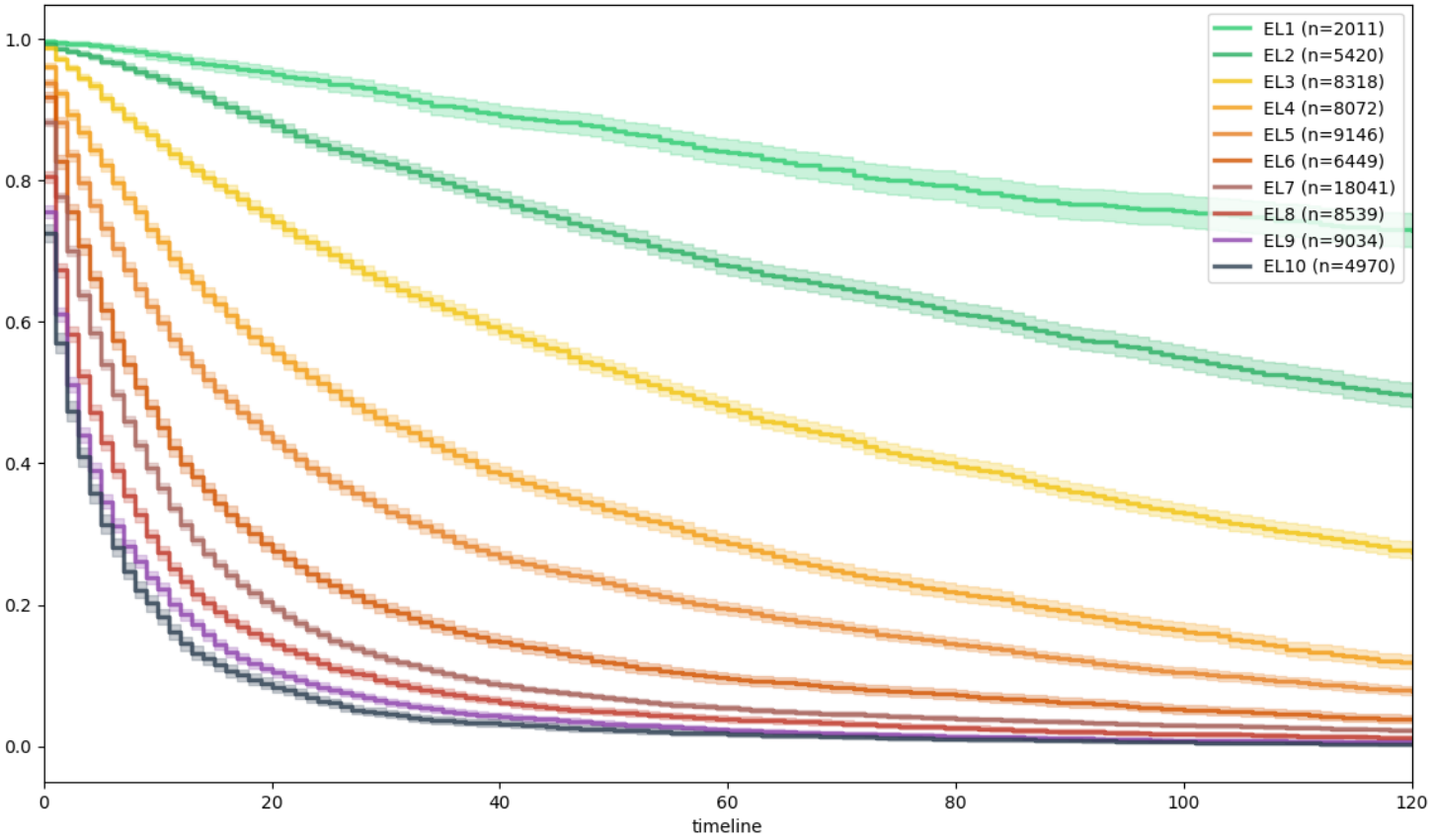


Figure 6: KM plot using the second optimal configuration on the unseen test data

Both KM plots are highly similar to each other, although the configurations are different. For both plots, the curves EL3 and EL4 show equal separation over time beyond the 40 month time point, however, the remaining consecutive pairs of curves, such as the curves representing EL1 and EL2, do not. Thus the KM plots could be further improved.

Although scikit-learns permutation_importance function was computationally prohibitive, an alternative estimate of feature importance was obtained by analysing the split count and mean depth of each feature. The table below presents feature importance rankings from most to least important for the model with hyperparameter values set to the second optimal hyperparameter configuration.

feature	split_count	mean_depth
Tumor Size Over Time Recode (1988+)	109	4.366972
Age recode with <1 year olds and 90+	102	4.656863
Histology recode - broad groupings	62	4.451613
Diagnostic Confirmation	41	4.585366
SEER Combined Mets at DX-liver (2010+)	40	3.975000
SEER Combined Mets at DX-bone (2010+)	36	4.000000
Site recode - rare tumors	27	5.074074
Regional nodes positive (1988+)	24	0.500000
Mets at DX-Other (2016+)	24	6.000000
Total number of in situ/malignant tumors for patient	23	5.434783
Regional nodes examined (1988+)	22	2.954545
SEER Combined Mets at DX-brain (2010+)	22	5.181818
SEER Combined Mets at DX-lung (2010+)	21	6.000000
Primary Site - labeled	17	5.470588
First malignant primary indicator	16	6.375000
Grade Recode (thru 2017)	12	4.916667
AYA site recode 2020 Revision	12	6.000000
Year of diagnosis	10	5.300000
Tumor Size Summary (2016+)	6	6.333333
Sex	5	6.400000

Table 5: Table showing the top twenty covariates with the highest combination of split count and mean depth.

The top two covariates "Tumor Size Over Time Recode (1988+)" and "Age recode with <1 year olds and 90+" were used in splits significantly more than the remaining covariates.

To investigate the efficacy of a deep learning model compared to a shallow learning model, the DeepHit model will be implemented and evaluated in the following sections.

7 DeepHit

DeepHit was selected because it predicts a full survival probability distribution rather than a single risk score [12]. This enables a more fine-grained classification system, in which patients are assigned to survival probability brackets based on their estimated long-term outcomes.

Since DeepHit requires all inputs, including categorical columns, to be numeric, label encoding was implemented. Encoding unknown values into the mode of the known values was considered, but discarded due to the fact that this method misleads the model about what was known. Instead, unknown values were given their own unique encoding. Label encoding was used instead of hot encoding to allow for scikit-learns permutation_importance function to

be run within a reasonable time. This is because hot encoding would significantly increase the dimensionality of the dataset, not only causing significantly slower optimisation times, but would also lead to the permutation_importance (sklearn's function for determining feature importance) function taking too long to run.

As the output of DeepHit is a series of probabilities for each patient, each index representing a time point and the value representing the probability the patient survives up to that time point, risk labelling was performed based off of the average survival probability of the probability series predicted for each patient. The average probability was chosen instead of the probability at the time index near a given time point to enable representation for all curves. This is because the probabilities for each patient for any time point fall into the same probability decile. Figure 7 is provided below as an example.

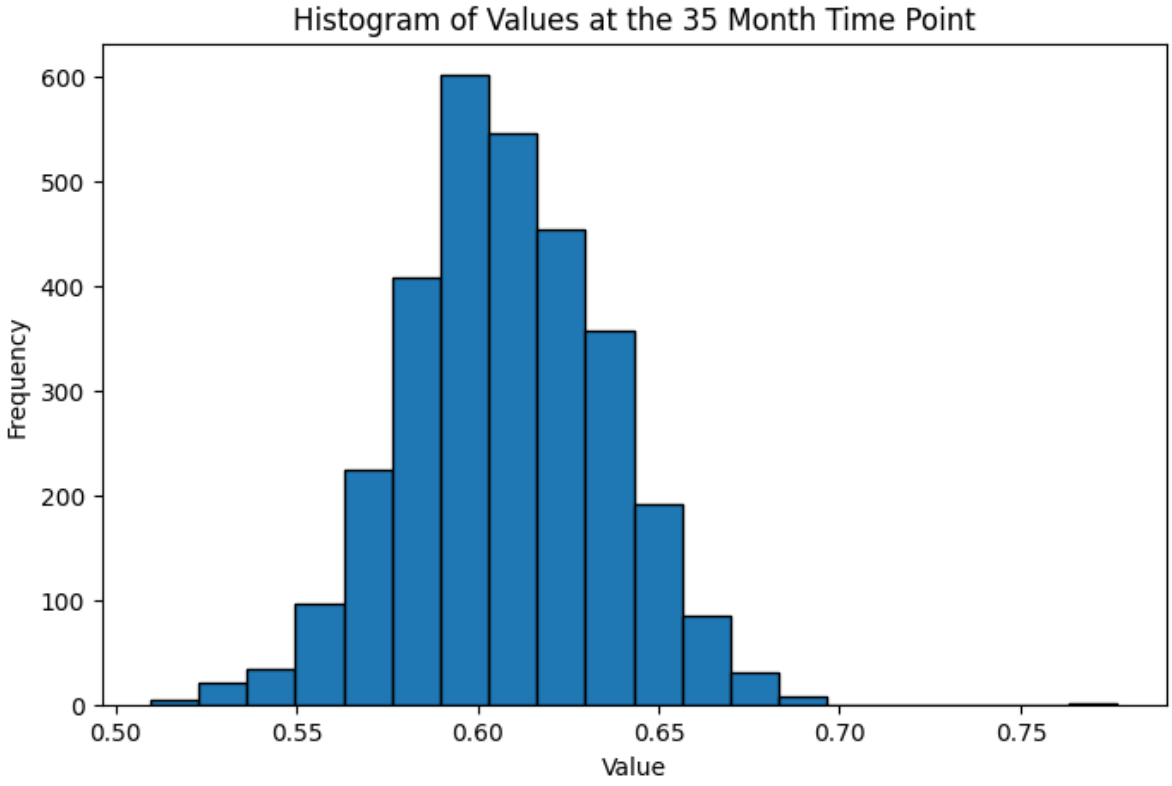


Figure 7: Histogram of the DeepHit output probabilities assigned to each patient at the 35 month time point. The histogram indicates that all patients would only be partitioned into two decile brackets ([0.5,0.6],[0.6,0.7]) leading to a KM plot of only two curves rather than 10, one for each decile.

Assigning patients a risk label based off of the cumulative sum of the probabilities up to a specific index was considered, however, taking the mean of the survival probabilities for each patient proved to provide better representation for each survival curve in the KM plot (representing a risk label).

7.0.1 DeepHit Hyperparameter Tuning

The table below shows the optimised values for the DeepHit model, along with the implications following the optimised values.

Hyperparameter	**Value**	**Comment**	**Justification**
Learning Rate (lr)	[0.001,0.1]	Controls the step size in which the model takes when updating its weights during training.	The upperbound of 0.1 was chosen since higher step sizes would cause instability. The lower bound of 0.001 was chosen since it is sufficiently close to 0, going lower by a factor of 10 would lead to compute time that is too high.
Alpha (α)	[0.001,1]	Weight for the likelihood loss.	The lower bound of 0.001 for alpha has been applied for the same reason as that of the learning rate. The upper bound of 1, the highest value alpha can be has been provided to maximise the range of the search space.
Sigma (σ)	[0.001,1]	Weight for the ranking loss.	This search space has been given for the same reason as that of alpha.
L transformation dropout	[0,0.99]	Controls the fraction of neurons randomly deactivated during each training pass, reducing overfitting.	The lower bound of 0 will allow the optimiser to compare model performance against one without dropout. Setting the upper bound to 0.99, a value close to the maximum possible dropout value of 1 will provide the widest feasible search space.
num_nodes	[128,64], [256,128,64], [256,128,64,32], [512,256,128]	Controls the number of layers and the number of neurons in each layer.	The discrete search space given is used to compare the effects of increasing complexity through depth and node size.

Table 6: Table to illustrate the DeepHit model hyperparameter search space.

8 DeepHit Evaluation

After performing Bayesian optimisation using ten curves, the score of the best configuration was 1000, representing the penalty score. This score implies that for evaluation during Bayesian optimisation, the optimiser was unable to determine a configuration that did not not lead to curves intersecting or the order between risk groups to be broken, leading to the penalty of 1000. The KM plot of the optimised configuration is shown below.

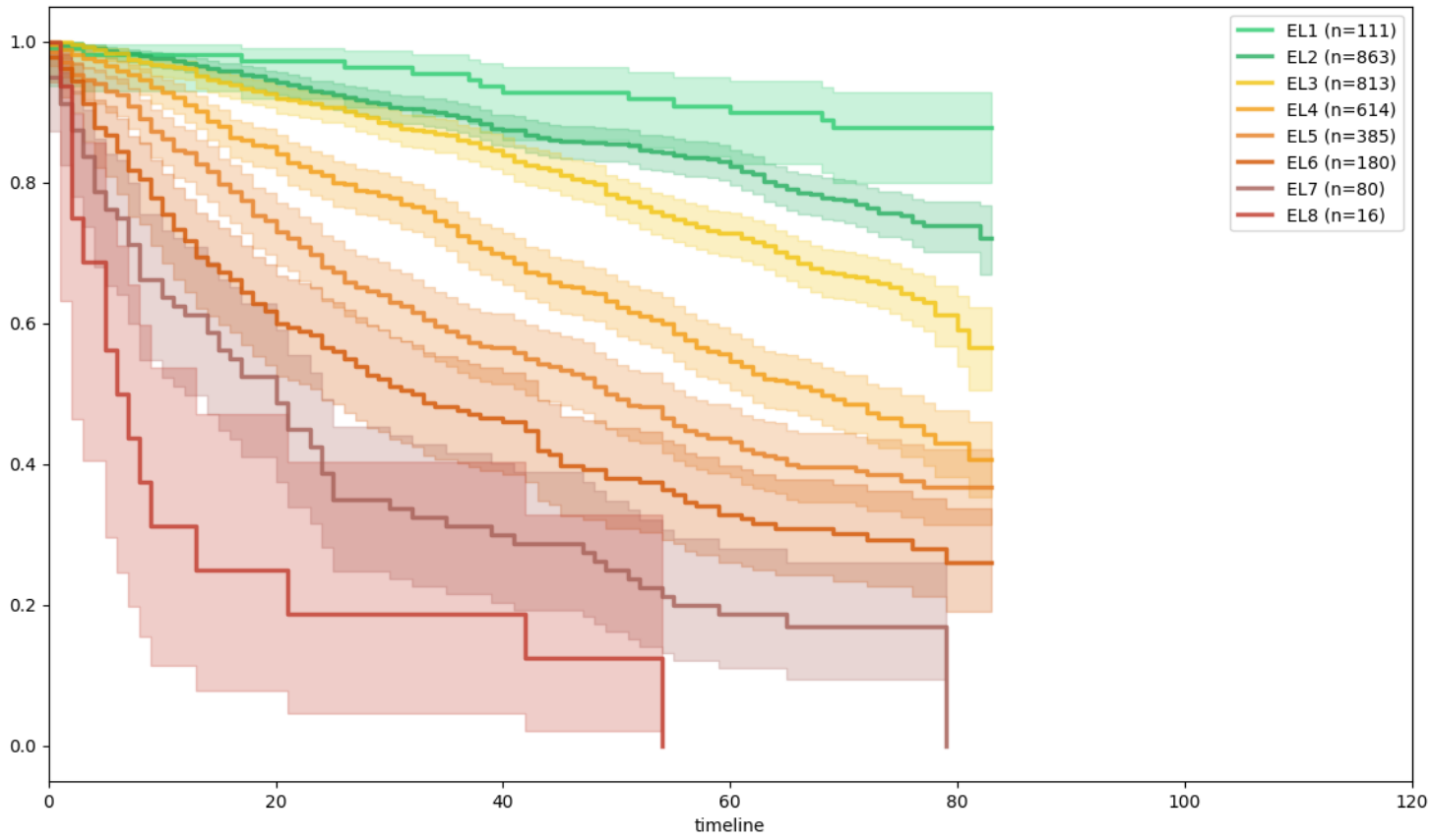


Figure 8: KM plot using classification from the DeepHit hyperparameters

The reason why curves EL8 stops before the other curves is because there are no patients classified as EL8 that have lived long enough to reach the 60 month time point. There is insufficient patient records to represent each of the ten risk labels. This is because the DeepHit model did not identify a sufficient number of patients belonging to the EL8 risk group. Curves belonging to EL9 and EL10 are also missing from this plot since no patients were identified to belong to the risk group.

To ensure better representation for each group, the same binning approach for the Random Survival Forest model will be implemented, with log hazard ratios replaced by the mean probabilities.

To ensure better representation amongst curves, only 6 curves will be used, as the number of curves are still higher than that of the Tumour Node Metastasis (TNM) staging system, and so should provide more specific representation of patient risk, with two more patient risk profiles.

Both optimised configurations led to a KM score of 1000. The table below show the configured parameters and their interpretations.

Hyperparameter	**Config 1 Value (4sf)**	**Config 2 Value**
dropout	0.3985	0.3985
lr	0.0192	0.0192
alpha	0.7799	0.7799
sigma	0.5973	0.5973
num_nodes	256,128,64	256,128,64

Table 7: Comparison of Two DeepHit Hyperparameter Configurations with Interpretations

The figure below outlines the KM plot following hyperparameter optimisation, for each configuration.

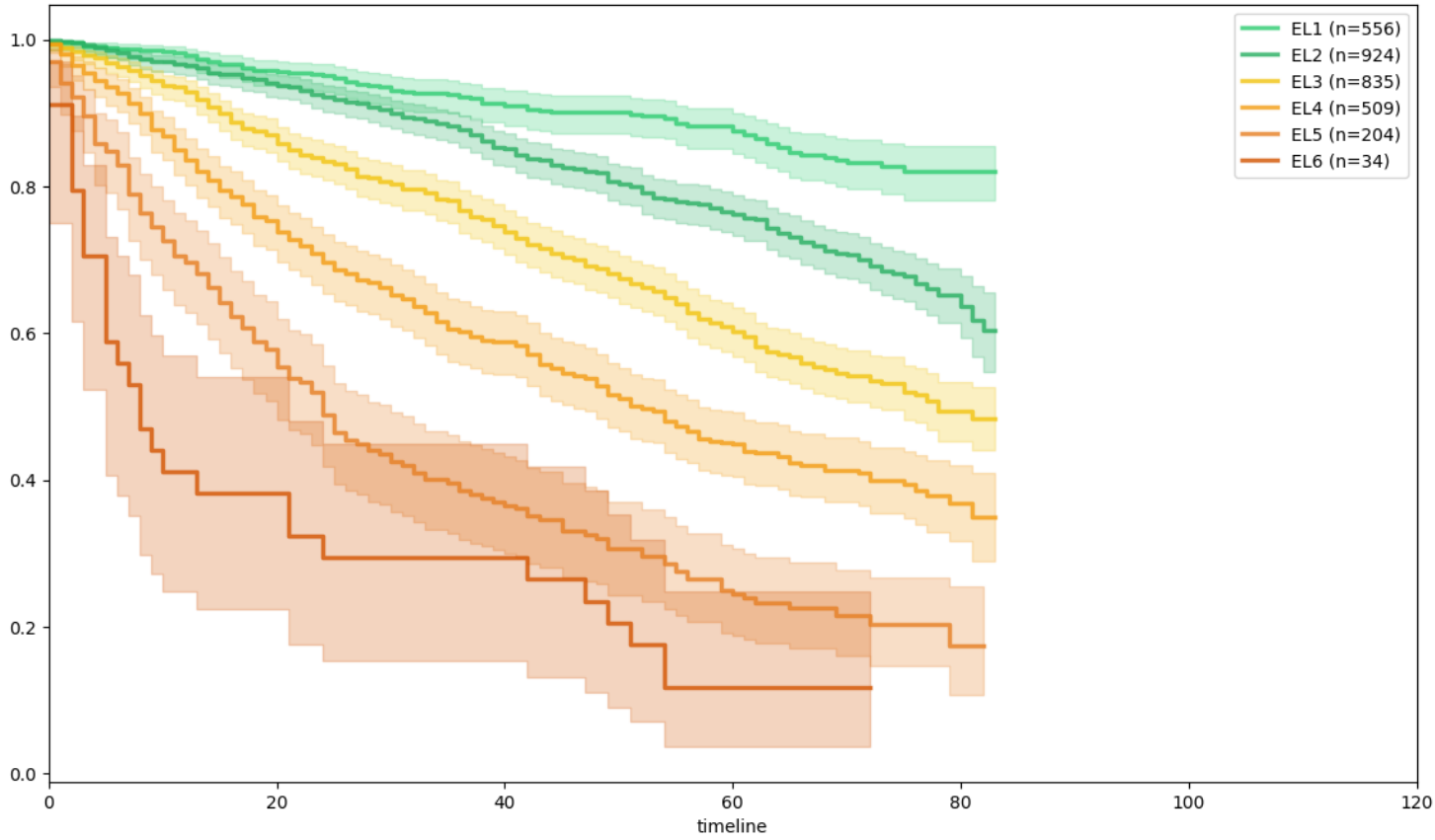


Figure 9: KM plot created using model predictions on unseen test data witb hyperparameters set to the first optimal configuration.

The KM plot above shows an intersection between the survival curves for EL1 and EL2 between the 0 and 5 month time points. This is the reason why the plot corresponded to a KM score of 1000.

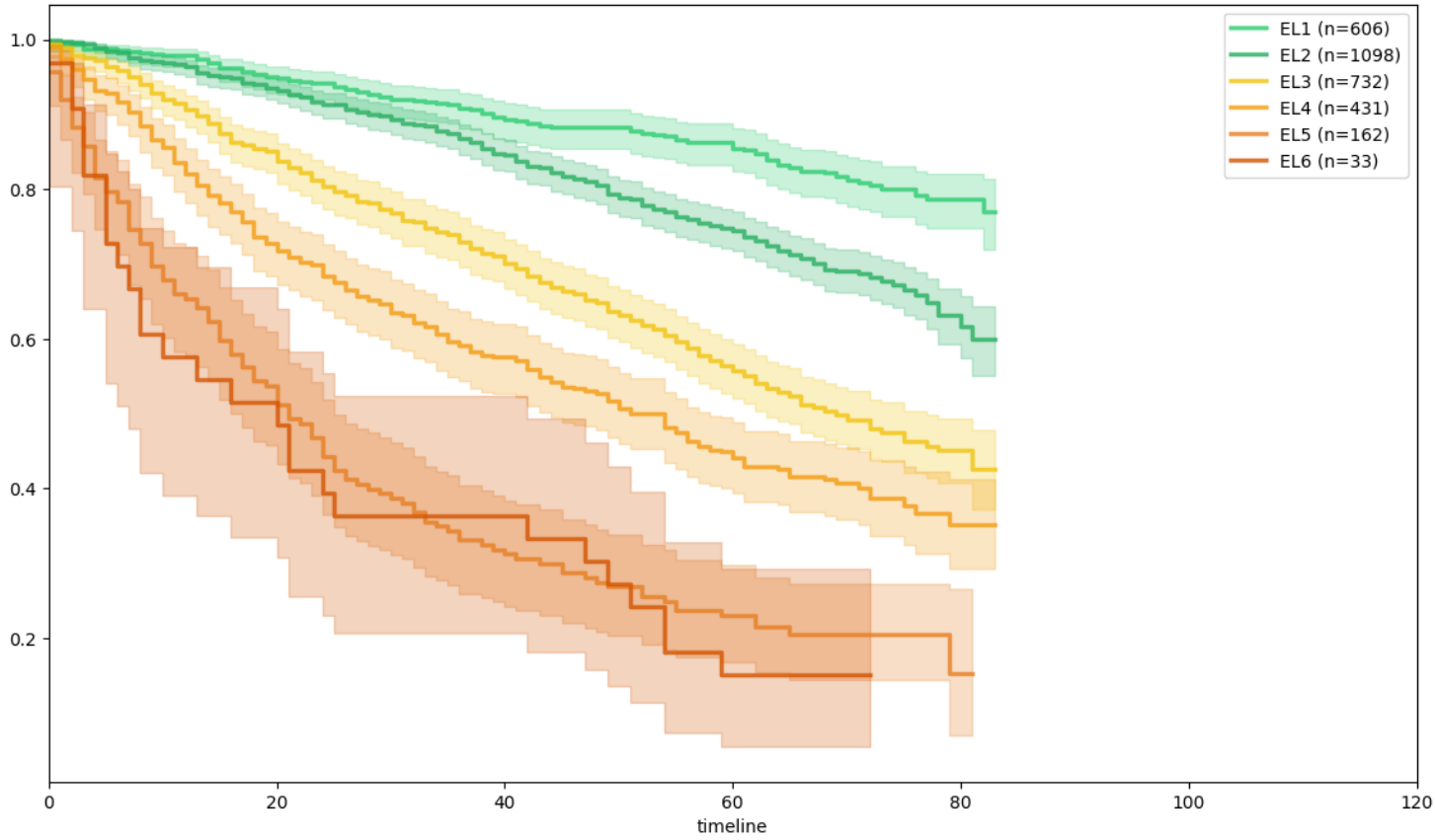


Figure 10: KM plot created using model predictions on unseen test data with hyperparameters set to the second optimal configuration.

The KM plot above shows an intersections between the curves representing risk labels for EL5 and EL6 (at time points 20, 32, 49), leading to a resulting score of 1000.

8.0.1 Analysing Feature Importance of The DeepHit Model

Feature importance and analysis of covariates following this was done in order to better understand areas of the dataset that are particulary influential in determining a patients risk, aiding in the optimisation of the next machine learning model.

To determine the feature importance on the DeepHit models predictions, sklearn's permutation_importance function was used. The permutation_importance works via measuring how much each feature contributes to the model's predictive performance. This is done by evaluating the baseline performance of the model (performance of the model on a dataset where no covariates are removed). Then

one covariate has their values shuffled, destroying relationship between that covariate with other covariates as well as the survival duration and event status. The performance drop is then measured, with all measurements determining the order of feature importance. Attaining the permutation importance for the DeepHit model involved implementing a separate KM scorer function to take in the required arguments (estimator, X, y) in order to be passed as the scorer argument for the permutation_importance function. However, since all hyperparameter configurations lead to a KM score of 1000, all covariates were assigned a feature importance of 0, since when any feature is removed, the KM score for the model trained on the dataset with the removed feature still attains a value of 0. Although the c index is not representative of the KM score, determining which covariates improved the ability for the model to identify patient risk correctly would be informative in determining the most important covariates regarded by the model for reducing the KM score.

The table below displays the features that were most important to the DeepHit model with regards to improving the c index, from most to least important.

Feature	**importance_mean**	**importance_std**
Regional nodes positive (1988+)	0.032175	0.004895
Age recode with <1 year olds and 90+	0.026072	0.005517
Regional nodes examined (1988+)	0.013213	0.002233
Sex	0.012705	0.002845
Histology recode - broad groupings	0.010319	0.002116
Tumor Size Summary (2016+)	0.009021	0.003001
Tumor Size Over Time Recode (1988+)	0.006654	0.002671
Rural-Urban Continuum Code	0.005032	0.004248
AYA site recode 2020 Revision	0.003869	0.001948
Total number of in situ/malignant tumors for patient	0.002346	0.001327
Grade Recode (thru 2017)	0.002305	0.002448
Year of diagnosis	0.002114	0.002073
SEER Combined Mets at DX-bone (2010+)	0.001843	0.000713
SEER Combined Mets at DX-lung (2010+)	0.001773	0.000573
Marital status at diagnosis	0.001673	0.001892

Table 8: Top 15 most important features (permutation importance)

The importance_mean column and importance_std column stores the mean and standard deviations of the importance scores across multiple permutation repetitions. Similar to the ranking of feature importance for the Random Survival Forest model, the covariates "Age recode with < 1 year olds and 90+" and "Tumor Size Over Time Recode (1988+)" are amongst the most important features. There is a relatively significant decrease (0.002367) in the mean importance from "Tumor Size Summary (2016+)" to "Tumor Size Over Time Recode (1988+)", implying that the main covariates influencing the model accuracy are represented by the top six covariates in the table. Each of these six covariates are investigated further below. [18].

1. Regional Nodes Positive (1988+)

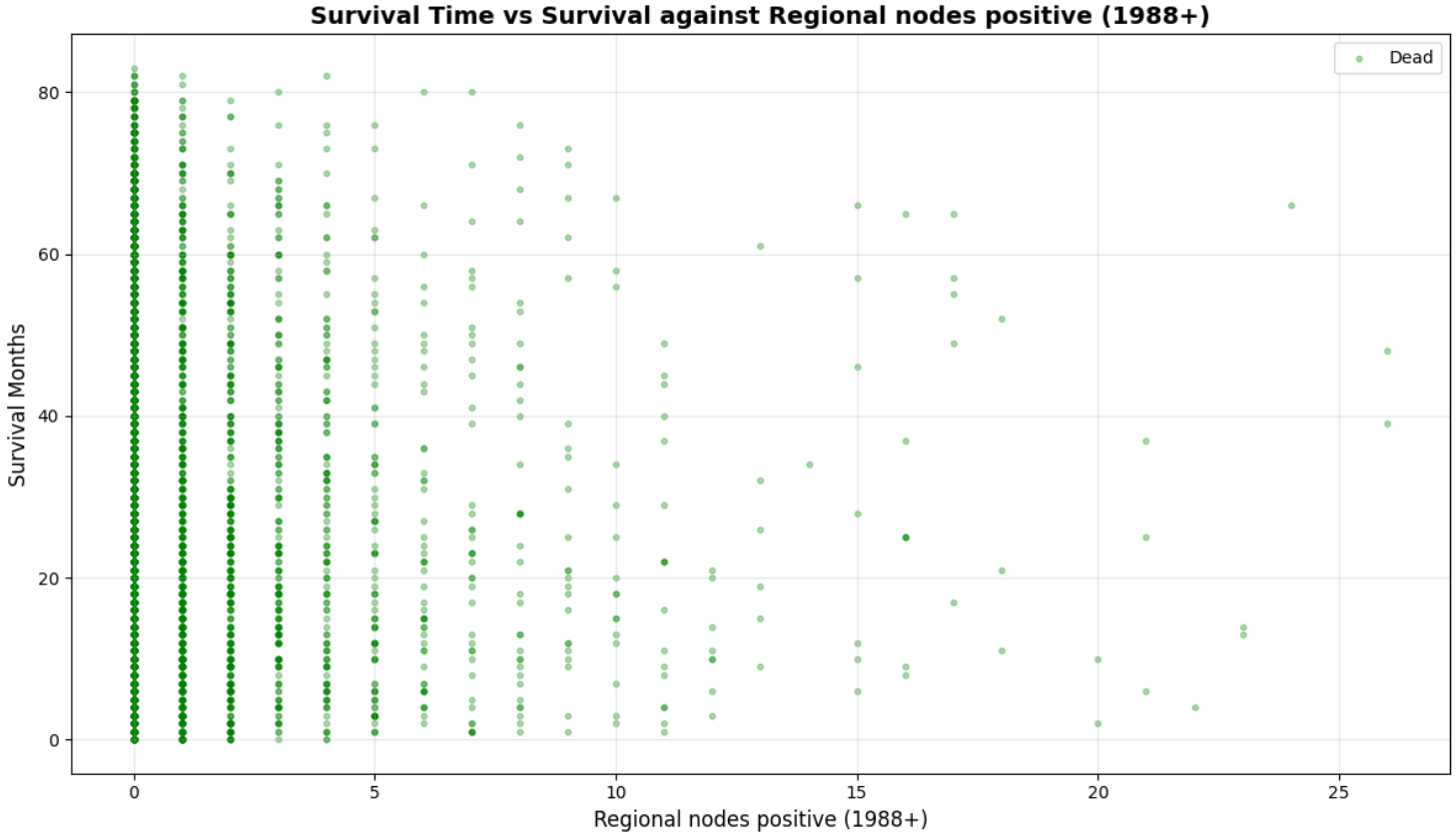


Figure 11: A scatter plot to show the relationship between the survival time and the number of positive regional nodes a patient has.

By looking at the maximum months survived for each regional positive node value, there appears to be a negative trend. However, there is insufficient support to show a strong negative association between the number of positive regional nodes and the months survived, although research supports this [18].

2. Age recode with <1 year olds and 90+

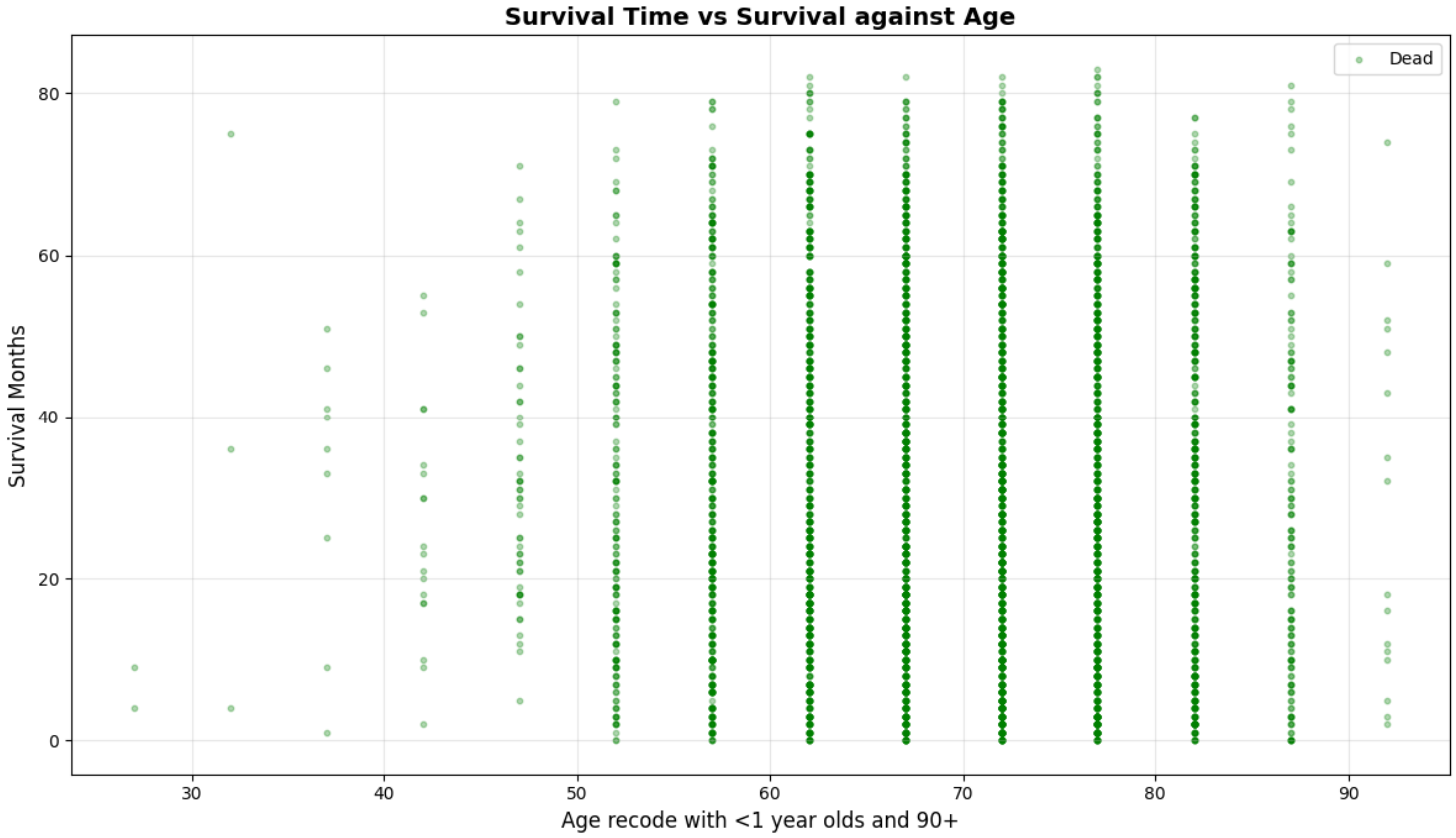


Figure 12: A scatter plot to show the relationship between age and survival time.

Although the age covariate was the second most important covariate, the scatter plot does not show an association between age and the months survived. Potentially, since DeepHit can identify non linear relationships, the model learns a range of possible survival durations a patient is expected to live given their age [17].

3. Regional nodes examined (1988+)

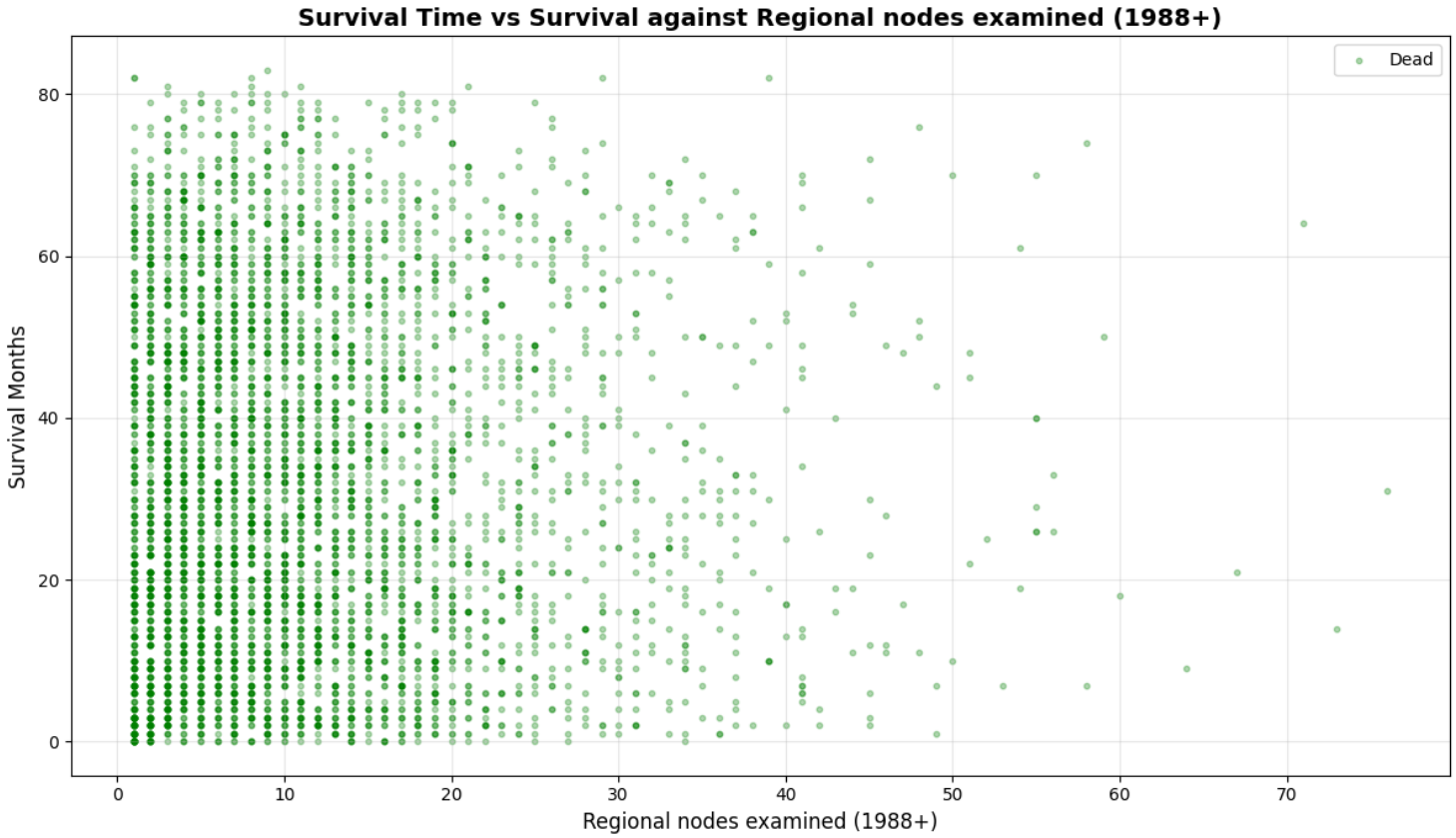


Figure 13: A scatter plot to show the relationship between the survival duration and number of regional nodes removed.

There does not appear to be any association between survival time and the number of regional nodes removed. Perhaps there is a threshold value for the number of regional nodes removed which the model uses to determine the patients survival probabilities.

4. Sex

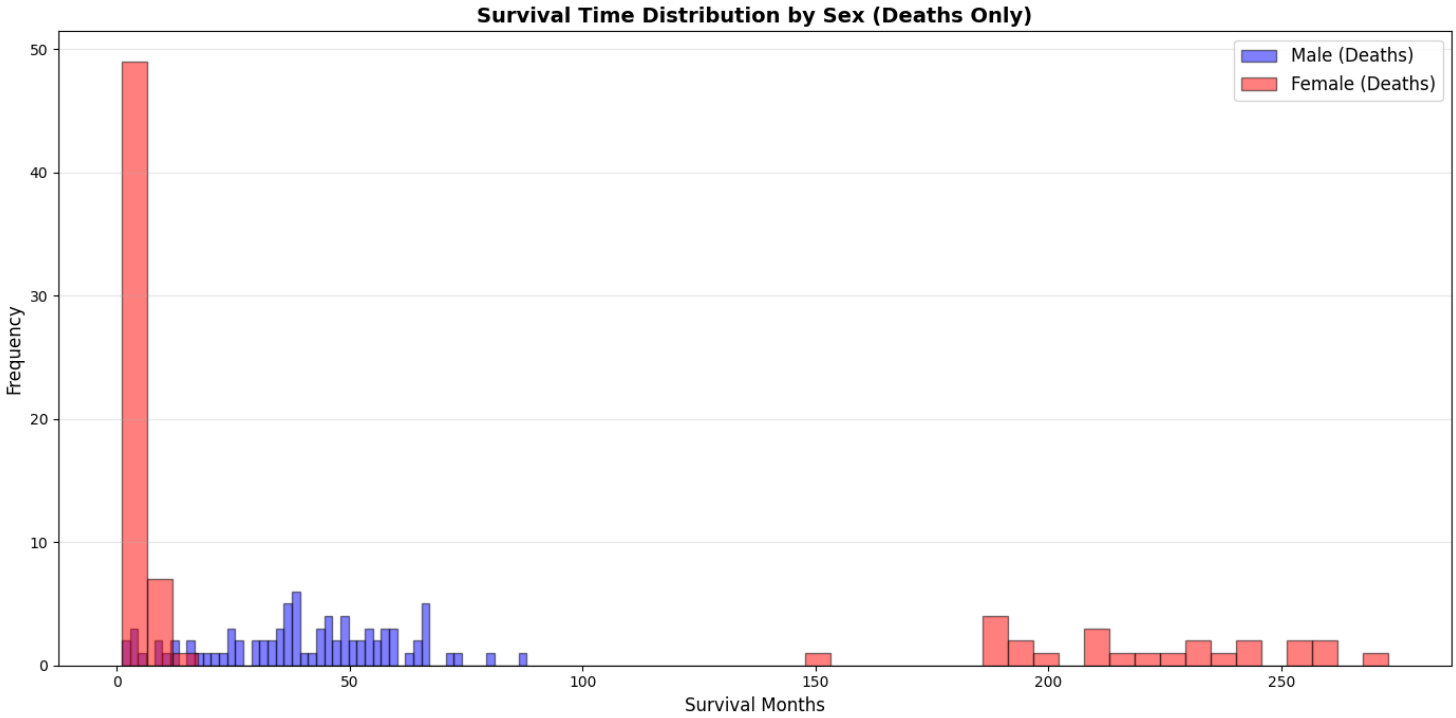


Figure 14: A histogram to show the frequency of each survival duration bin (out of only deceased patients, from a dataset with 15,000 rows due to removing rows with missing values), coloured by sex.

The plot shows a separation between the survival durations, with all deceased males having survived less than 100 months, whilst there exist a cluster of females who have survived beyond 175 months. The association between sex and survive duration, that is females with cancer being at lower risk than males with cancer, is supported in research [16].

5. Histology recode - broad groupings

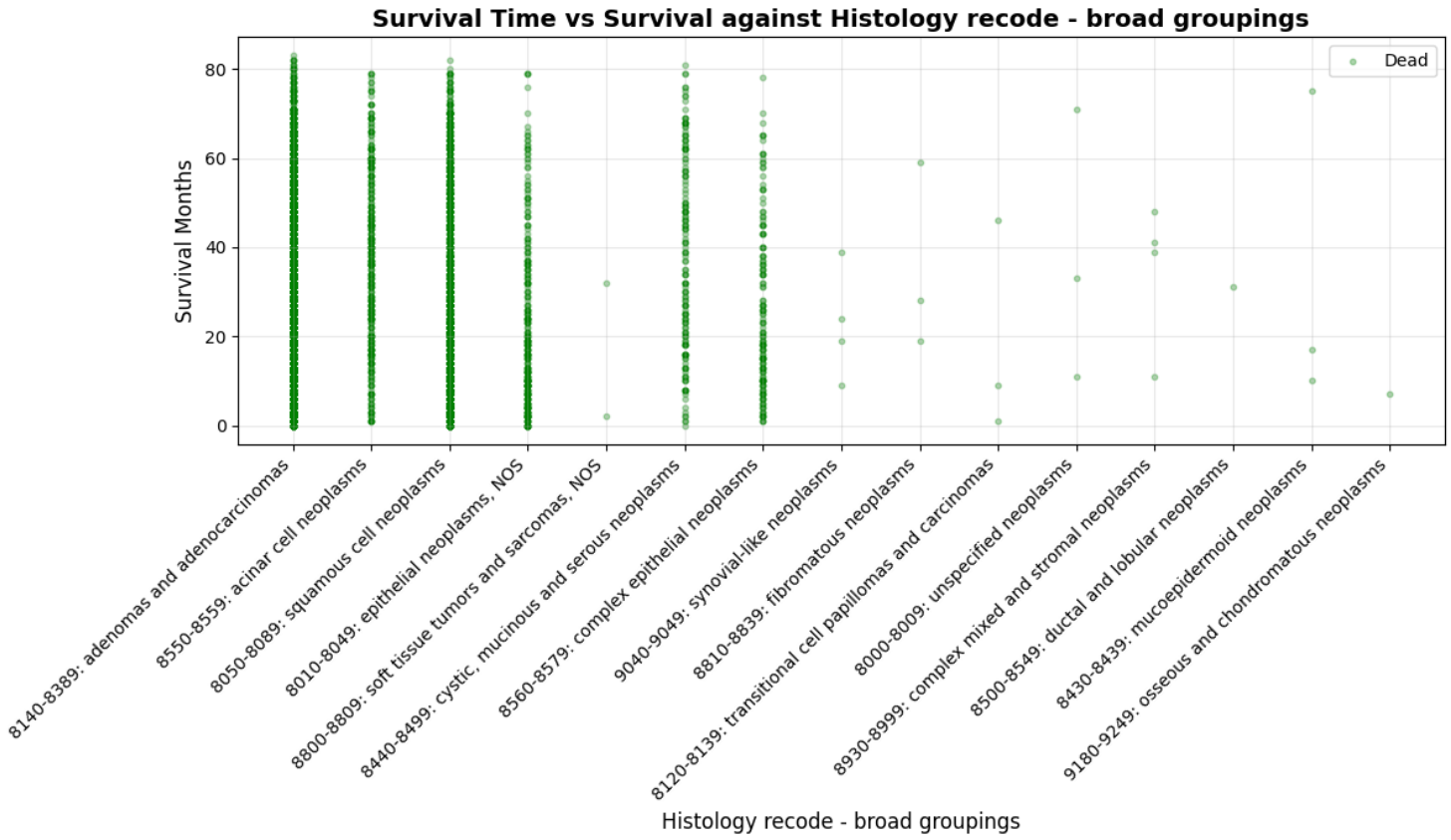


Figure 15: A scatter plot to show the relationship between survival duration and histology recode.

The scatter plot does not show an association between the histology recode and the survival duration, however, it does indicate which histology recode values are most common.

6. Tumor Size Summary (2016+)

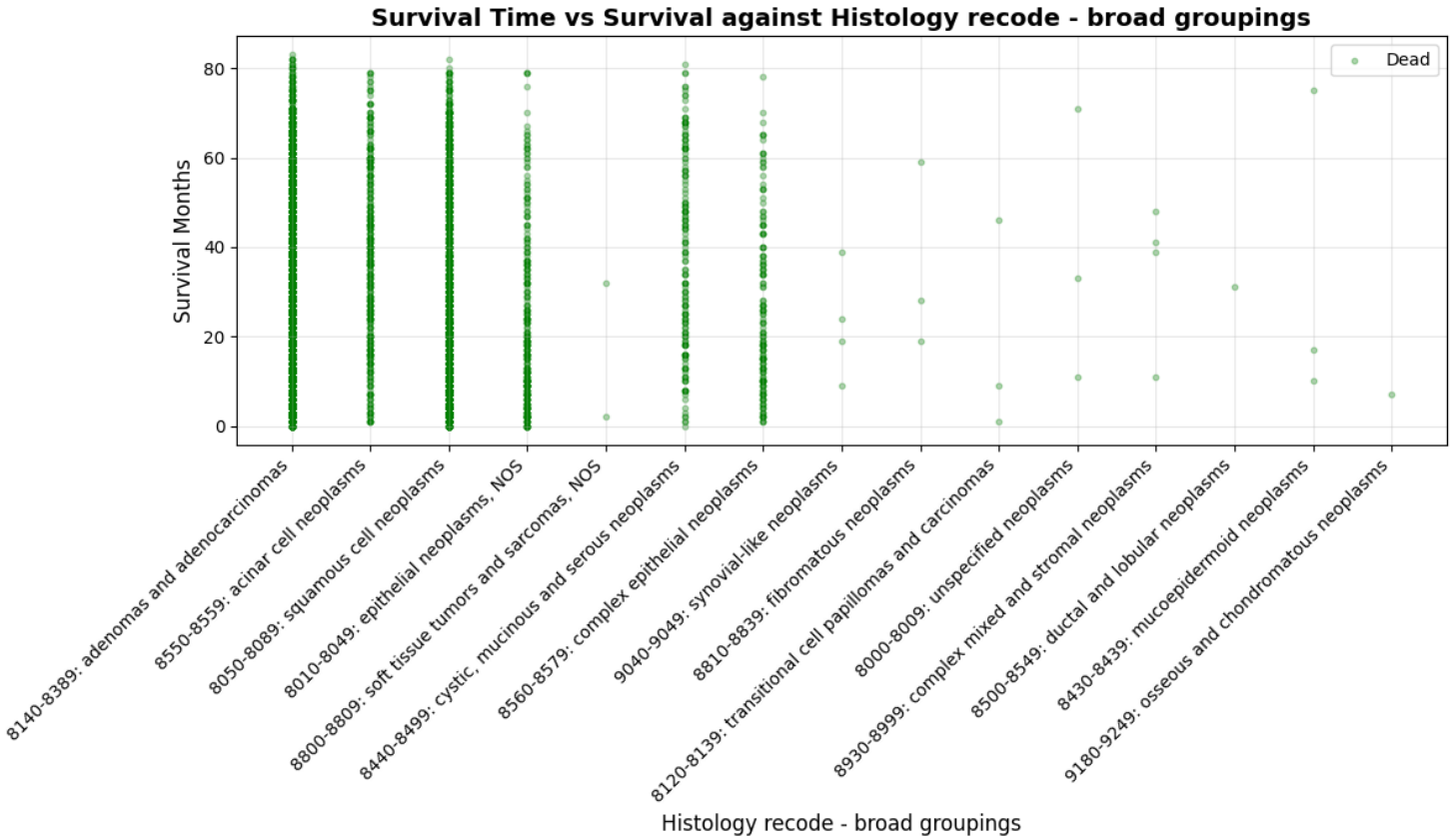


Figure 16: A scatter plot to show the relationship between survival duration and histology recode.

The scatter plot does not provide sufficient evidence to claim that there is a negative association between the tumour size summary (size of the tumour) and the survival duration of a patient.

9 XGBoost

Since the Random Survival Forest (RSF) model performed significantly better (lower KM score) than the DeepHit model, it was determined that increasing the model complexity does not necessarily lead to an improvement in KM score, that is, an improvement in the KM plots produced.

Therefore, the final model implemented, in an attempt to obtain a better KM score than RSF, was XGBoost with a survival:cox objective. XGBoost is

an ensemble learning model that uses boosting instead of bagging when aggregating decision trees, unlike with Random Survival Forest. Since XGBoost is a regression model, it is less complex than DeepHit. The main reason why the decision to implement XGBoost was made as opposed to continue with Random Survival Forest was due to easier interpretability of the XGBoost model via SHAP values, which will be discussed later. Easier interpretability will aid with improving the data through feature engineering. In addition, XGBoost has significantly more hyperparameters than RSF, allowing for further fine tuning.

9.1 Hyper-parameter Tuning

Bayesian optimisation was used. The table below illustrates the search space for each hyperparameter, as well as the hyperparameter descriptions and their justifications.

Parameter	**Value**	**Description**	**Justification**
objective	survival:cox	Specifies the learning task and corresponding objective function.	The survival:cox objective leads to the model outputting log hazard ratios, which is the same output as that of the Cox Proportional Hazard model used to generate the KM plot.
base_score	0.5	Initial prediction score of all instances (global bias).	This has been set to the default value of the XGBoost model to prevent loss and gradients from becoming unstable. If the loss is unstable, the XGBoost model predicts NaN on the test data, which will not lead to improvements made from the Bayesian optimiser as the configuration will be no better chosen than the original, leading to each chosen configuration to cause XGBoost to output NaN.
eta	[0.01,0.9]	Shrinkage (learning rate) applied to each new tree.	Values near 1 can overfit, since the dataset is large, a relatively high upperbound of 0.9 has been provided, to allow the possibility of faster runtime with similar accuracy. The value 0.01 was chosen as lower eta values are better prevent overfitting, having a lower bound of 0 may lead to errors if the optimiser chooses to configure the eta to 0, since this would lead to no learning.

booster	[gbtree,gblinear]	Which booster to use (gbtree, gblinear, dart).	Gblinear (linear boosting) has been considered since the numerical covariates in the dataset in theory have a linear relationship with survival duration, such as the number of regional nodes a patient has. gblinear is known to work best when the relationships between covariates and the truth label are linear. gbtree has been chosen due to the potential existence of heirachical and non linear relationships between the covariates and the truth label. dart (variant of gbtree) has been considered in the search space due to the dataset having a high number of missing values (noise), over 1 million rows (99% of the data) in the dataset contain a missing value. dart performs the best relative to the other boosting options for data with high noise. Dart is not used to prevent the model from producing NaN outputs if the dropout is too high.
colsample_bylevel	[0.5,0.8]	Fraction of columns out of all the columns to be considered for selection per level.	The lowerbound of 0.5 has been used as a rule of thumb to prevent underfitting cite, an upperbound of 0.8 as there is over 20 features and the dataset has high noise. Having 0.8 as the upperbound prevents the trees from memorising noise in the data rather than learning generalisable patterns.
colsample_bytree	[0.5,0.7]	Subsample ratio of columns for each tree.	The lower bound is set to 0.5 for the same reason as in the cell above. The upperbound of 0.7 has been set to prevent trees from being too similar, allowing for more patterns in the dataset to be recognised during the training process.
eval_metric	cox-nloglik	Evaluation metric for validation data.	cox-nloglik is the most appropriate for Cox survival objectives as it directly measures the survival function that the XGBoost model with objective survival:cox optimises.

feature_weights	–	Provides optional weights for each feature.	Set to none for now since it is not clear for now whether a specific feature requires more importance than the model will assign to it.
interaction_constraints	–	Determines constraints for feature interaction. That is, given a list of lists containing column indices, trees that use a particular column index can only use column indices in the belonging to the same list as the column index used.	This is currently set to none however feature interaction will be tested in the following sections.
max_bin	296	Controls the maximum number of bins used to discretise continuous features	The covariate with the highest number of unique values is 'Tumor Size Summary' which contains 296 different values, therefore keeping the maximum number of bins allows each value to be uniquely represented in XGBoost.
early_stopping_rounds	20	Stop after no improvement for N rounds.	A value of 20 has been chosen as it is large enough to prevent the model from stopping too soon and missing out from further fitting (such as a value of 5) but low enough to avoid extra compute time that does not improve the score (such as with a value of 100).
max_cat_threshold	[1,42]	Max categories considered for a split.	The lowest and highest feasible values of 1 and 42 (the highest number of unique categories belonging to a covariate in the dataset) has been given to provide the widest possible search space, this is to allow the bayesian optimiser to determine best fit.
max_delta_step	[1,10]	max_delta_step is the maximum change in weights that a single tree is allowed to contribute to the ensemble in one boosting step	A lower bound of 1 is chosen instead of 0 due to class imbalance within the dataset (50 times more patients have value "C34.1-Upper lobe, lung" than value "C34.8-Overlapping lesion of lung" for the "Primary site - labeled" covariate). The upper bound of 10 is chosen since a higher value would be the equivalent of not having a max_delta_step at all.
gamma	[0,5]	Minimum loss reduction required to make a split.	An upper bound of 5 is set since any value beyond 5 would be too strict [1].

grow_policy	depthwise	Tree growth policy.	Depthwise has been chosen instead of lossguide due to the dataset having a high number of missing values as well as skew, with lossguide, the trees are prone to investigating noise, which is unwanted.
max_depth	[2,10]	Maximum tree depth for base learners.	Controls model complexity; deeper trees can overfit, shallow trees may underfit.
max_leaves	[2,64]	Maximum number of leaves per tree.	Limits tree size; higher values allow more complex trees, but risk overfitting.
min_child_weight	[1,10]	Minimum sum of instance weight (hessian) needed in a child.	Due to the high noise in the dataset, an upper bound of 10 is used to prevent allow for more generalisable rules rather than specific ones to prevent splits that focus on noise rather than meaningful relationships
monotone_constraints	None	Enforce monotonicity for covariates within the model, if a constraint of +1 is assigned to a covariate then the model should never predict a higher risk for a lower value of that covariate	No monotone constraints have been provided to allow the model to freely determine order for particular covariates.
n_estimators	[50,1000]	Number of boosting rounds for each tree.	A high upper bound was provided since a higher number of boosting trees will tend not to overfit but rather plateau in score.
num_parallel_tree	[1,3]	Number of trees added per boosting round	The upperbound has been set to 3 to avoid extra computing time, especially when num_estimators (number of boosting rounds) is configured to a high value (n_estimator is multiplied to this hyperparameter to determine the total number of trees).
reg_alpha	[0,10]	L1 regularisation term on weights	This allows the model to filter out the irrelevant features by driving weights exactly to 0. The upper bound of 10 was chosen to allow the full range of strength of L1 regularisation.

reg_lambda	[0,10]	L2 regularisation on weights	L2 shrinks all weights evenly, thus being a smoother alternative to L1 regularisation, a wide search space has been given to allow the optimiser to decide between L1 and L2 regularisation.
sampling_method	'uniform'	Controls how rows are selected when subsampling the training data for each boosting round.	Gradient based sampling (the only other option) is not considered as this sampling method requires GPU, which is unavailable.
scale_pos_weight	[1,10]	Used to control the balance of positive and negative weights	The upperbound is set to a high value of 10 to account for the imbalance in the dataset, a high value should reduce the the negative impact of skew between patients with a survival duration less than ten months and those with a survival duration of more than 50 months, by providing higher weighting to patients with survival durations that are underepresented. The lower bound of 1 is used to allow for a configuration with no weighting.
subsample	[0.5,1]	Subsample ratio of the training instances	A lower bound of 0.5 has been given due to the class inbalance within the dataset, selecting less than half of the data to train a tree may lead to the tree not learning the entire distribution of the dataset.
tree_method	['hist','approx','auto']	The hyperparameter determines how XGBoost builds each decision tree. The 'approx' method uses histogram approximation to find split points, the hist method uses an optimised histogram based method.	The value "Exact" was not considered in the search space due to the high runtime and minimal improvements in accuracy from "hist", since the "exact" method will evaluate a split for each unique value for each row in the dataset, as the training dataset has over 800,000 rows, the runtime for this method will be too long. The "auto" method was also not selected in order to prevent the "exact" method from being used. Otherwise, all remaining possible methods are considered in the search space.

Table 9: XGBoost model parameter search space

9.1.1 Evaluating the XGBoost Model

The Bayesian optimiser was run for 50 evaluations, providing enough evaluations for the optimiser to find the optimum hyperparameter configurations without excessive compute time. The optimised hyperparameter configurations produced Kaplan Meier scores of 0.393 and 0.387 (3sf).

The table below shows the corresponding parameter configuration along with their interpretation.

Hyperparameter	Config 1 Value	Config 2 Value	Interpretation
eta	0.3854	0.4085	The learning rates had moderate values for gradual learning.
booster	gblinear	gblinear	This implies that the relationships between covariates and survival are predominantly linear.
colsample_bylevel	0.6570	0.6778	The show moderate feature sampling for regularisation.
colsample_bytree	0.5298	0.5162	This means that each tree only sees around half of all the available features, implying feature redundancy.
max_cat_threshold	15	16	Selecting only 15 or 16 categories for each split imply that not all categories were equally informative.
max_delta_step	3.0937	3.1794	These values imply the categorical covariates in the dataset have a class imbalance.
gamma	4.1309	4.2126	This implies that the model needs strong evidence before making a split.
max_depth	4	4	Since 4 lies closer to the lowerbound of the search space, the value implies that the survival relevant patterns within the dataset are relatively low.
max_leaves	26	32	The max depth was 4 for both configurations, which meant that the maximum number of leaves was 16, thus these values are irrelevant.
min_child_weight	5	5	These values imply that the dataset is noisy enough to require evidence before splitting.
n_estimators	252	254	These values imply that the model needs significantly more boosting rounds than the Random Survival Forest needed trees.
num_parallel_tree	2	2	This value being the mid point of the search space implies that computational efficiency was balanced.
reg_alpha	1.5512	1.5072	This implies that some covariates were eliminated from the model, implying that the model identifies some covariates as redundant.
reg_lambda	4.4978	5.0820	These values imply that strong L2 regularisation was required, which implies that most covariates needed their influence on patient risk dampened.
scale_pos_weight	6.8183	7.2623	These values imply that class imbalance does exist. As higher importance to under represented groups were given.
subsample	0.9220	0.9292	These values imply that near full dataset sampling was optimal.
tree_method	auto	auto	No significant interpretation can be made.

Table 10: Comparison of Two XGBoost Hyperparameter Configurations with Interpretations

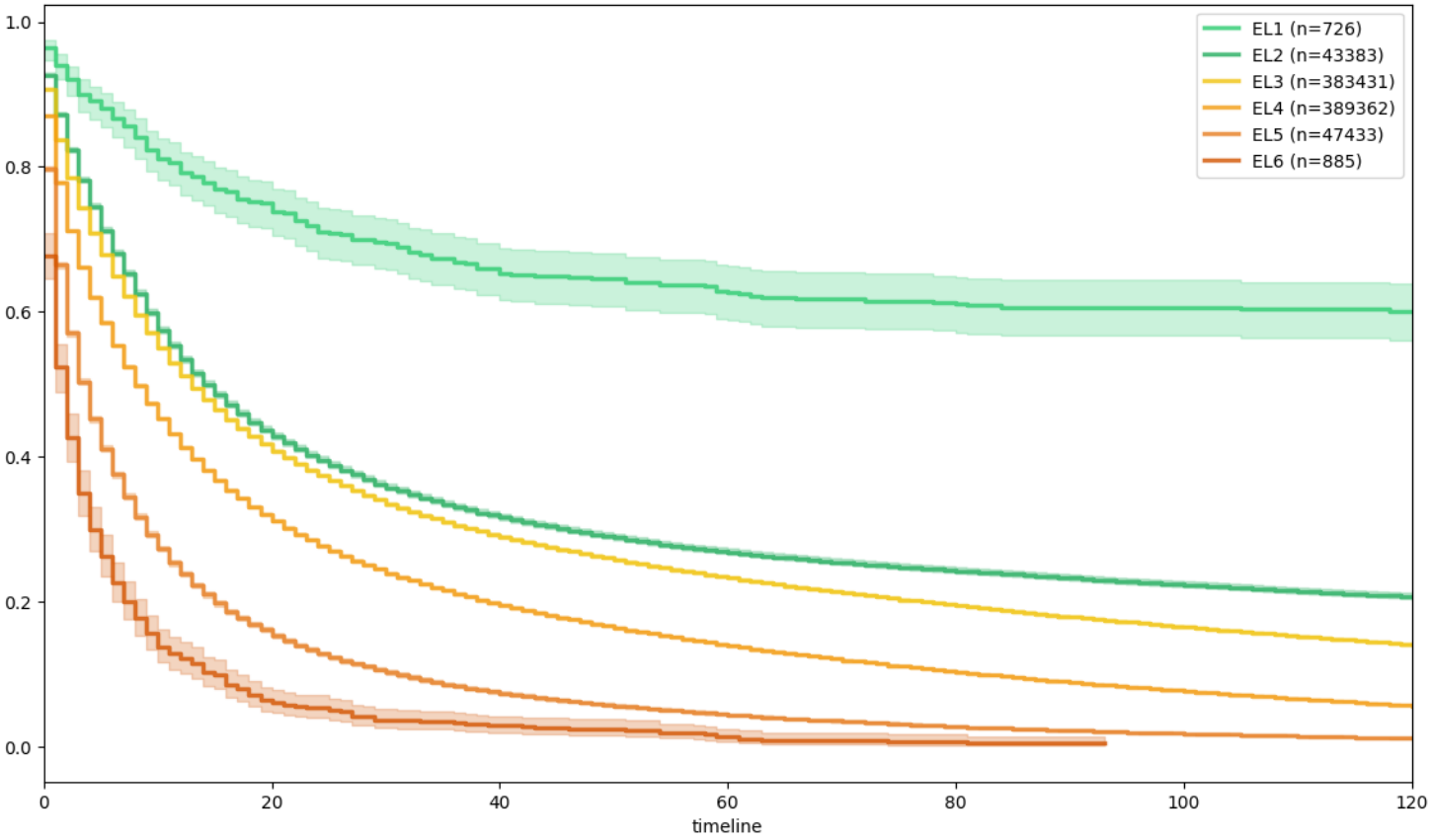


Figure 17: KM plot produced using the first configuration, corresponding to the model KM score of 0.393

The following KM plot was produced:

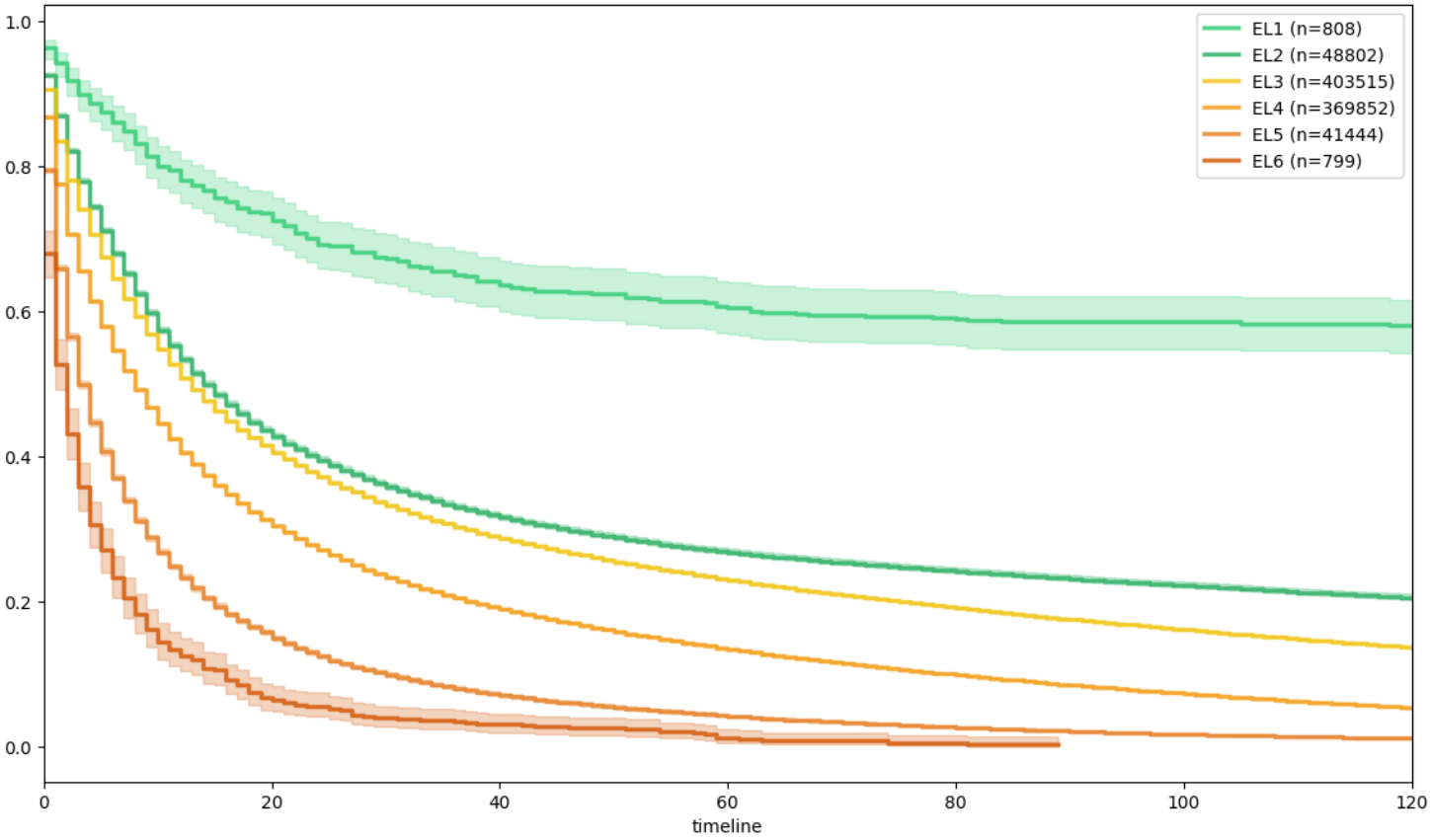


Figure 18: KM plot produced using the second configuration, corresponding to the model KM score of 0.387 (3sf)

The KM plots show unequal separation, as well as a disparity in representation (patient size) for each survival curve. However, both configurations produced highly similar KM plots (classification was done for unseen test data), despite their values being different. This proves that the optimiser has reached a stable configuration within 50 evaluations.

10 Using SHAP Values to Understand XGBoost Model

SHAP values were derived in game theory to represent the contribution of each player, in the case of machine learning, covariates replace the concept of players. SHAP values can be used to determine the direct impact each covariate has on the XGBoost model predictions.

Given input x and model f, SHAP value ϕ_i for feature i is defined as

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(n - |S| - 1)!}{n!} \left[f_x(S \cup \{i\}) - f_x(S) \right]$$

where:

- S is a subset of features excluding i.
- $f_x(S)$ is the value of the model prediction for instance x with features in S set to their values in x and other features marginalised.

To compute $f_x(S)$, the expected value of the model's output with respect to the missing features is calculated, the notation being:

$$f_x(S) = \mathbb{E}[f(x) \mid x_S]$$

where x_S is the set of features in S.

10.1 Findings from SHAP plot

To better understand the model and the key covariates that influence it, a beeswarm plot using SHAP values (which measure impact each covariate has on model risk predictions for each patient). SHAP values are in the form of a matrix with the same number of columns as the number of covariates and the same number of rows as the number of patients) will be implemented.

Each point on the beeswarm plot represents the impact on the output log hazard ratio from a covariate (each covariate has its own row). The covariates in the plot are ranked by feature importance, with covariates of highest importance being at the top. Colours closer to red indicate that the value of the covariate for that patient is high relative to other patients. Points on the x axis above 0 signify that the covariate value increases the risk of the patient, whilst the converse applies for points below 0 on the x axis.

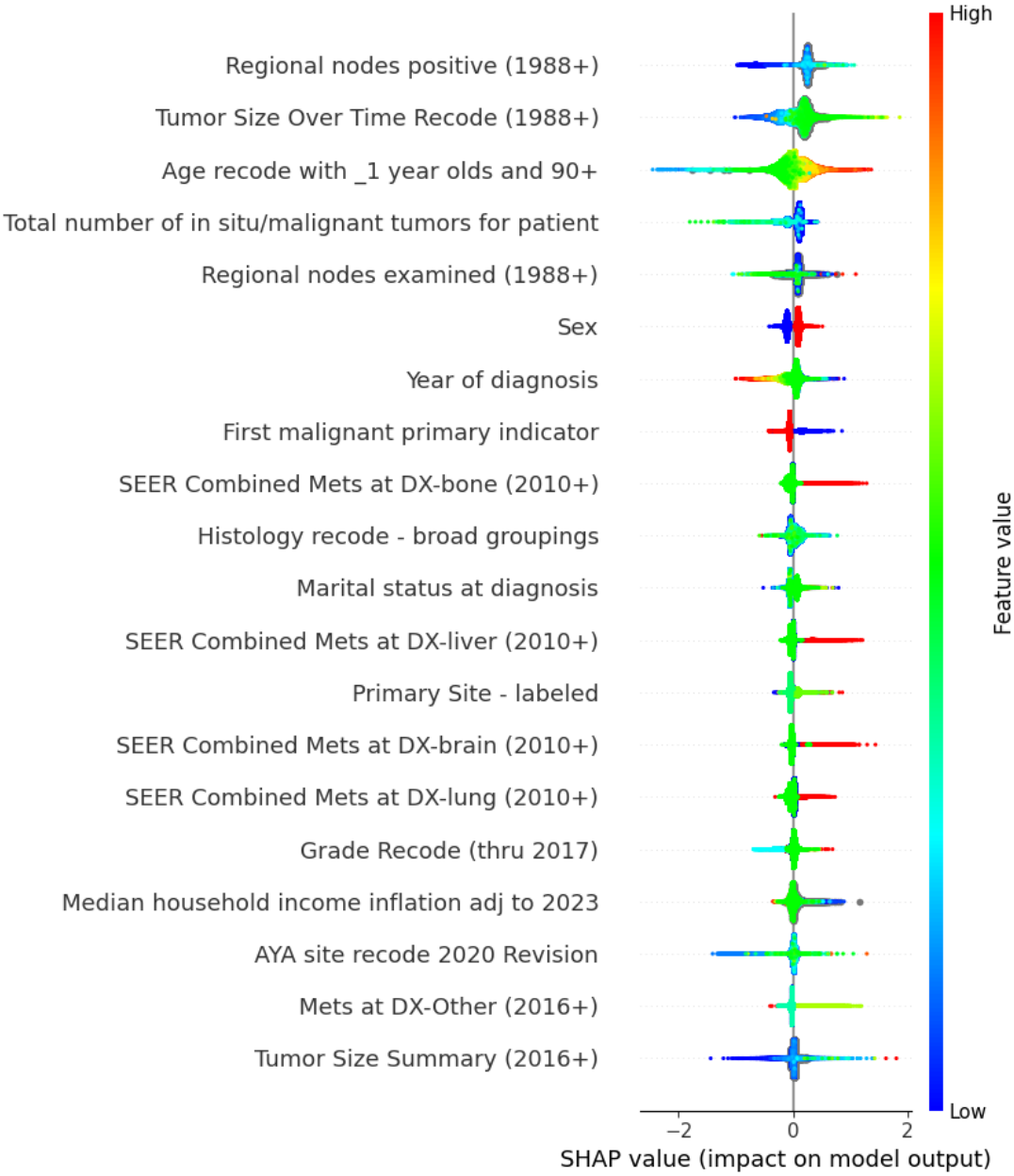


Figure 19: Beeswarm plot for XGBoost model using SHAP values.

The SHAP plot indicates that the XGBoost model percieves males with lung cancer to be at higher risk than females with cancer, this association is supported by the exploratory analysis done whilst investigating the top six most important

features for the DeepHit model. The plot also indicates that the model finds the site of the cancer ("Primary Site - labeled") to be influential in determining patient risk, the paper attached here [28] supports this finding. Some covariates such as "Total number of in situ/malignant tumors for patient" and "Tumor Size Summary (2016+)" do not show a trend between size and risk. Since these covariates belong to the top twenty most important features, there is reason to believe that the impact of these covariates on model prediction depend on the values of other covariates. This theory will be investigated in the following section.

11 Clustering for Hidden Covariate Relationships

To further reinforce the ideas mentioned in previous sections, such as there being only one dominant group of regional node values and the potential undetected cluster of patients pulling the interquartile range for the tumour size over time values (within the exploratory data analysis section), linkage clustering will be used to identify groups of patients, with the aim of determining hidden feature interaction that can be used to improve XGBoost model performance. Agglomerative clustering was used instead of partitioning clustering in order to avoid the requirement of having to specify the number of clusters. Average linkage was used instead of Single Linkage clustering to avoid chaining. The distance metric chosen was the Gower distance as this method does not assume order for categorical covariates, unlike other metrics such as the euclidean metric which would involve encoding labels into numbers. The Gower distance does this for categorical values by setting the distance to 1 if the values are different and 0 if they are the same. A more rigorous definition of the Gower distance is provided below:

$$[H]D_{\text{Gower}}(x_1, x_2) = \left(\frac{1}{p} \sum_{j=1}^{p} s_j(x_1, x_2) \right)$$

$$\begin{aligned} s_j(x_1, x_2) &= \delta(x_{1j}, x_{2j}) && \text{Categorical} \\ &= \frac{|x_{1j} - x_{2j}|}{\text{Range}_j} && \text{Numeric} \end{aligned}$$

Where $Range_j$ represents the difference between the highest and the lowest values for covariate j, and $\delta(x_{1j}, x_{2j})$ is a function that outputs 0 if $x_{1j} = x_{2j}$ and 1 otherwise.

Since scipy did not have an inbuilt submethod for computing the Gower distance within the scipy.linkage() method,the gower distance function was implemented from scratch and passed to the scipy.linkage() function.

To determine distance from a dataset that includes categorical variables, the Gower distance is used. An advantage of using Gower distance is that all distances are normalised via the covariates range. Because not all patients could be sampled (this required 2TiB of RAM), a random sample of 10,000 patients were taken. Since linkage cannot be performed with missing values, the dropna() method was used on the 10,000 patient subset to remove rows with missing values. This let to the subset containing 3000 rows (patients). Below is the dendogram resulting from agglomerative linkage clustering:

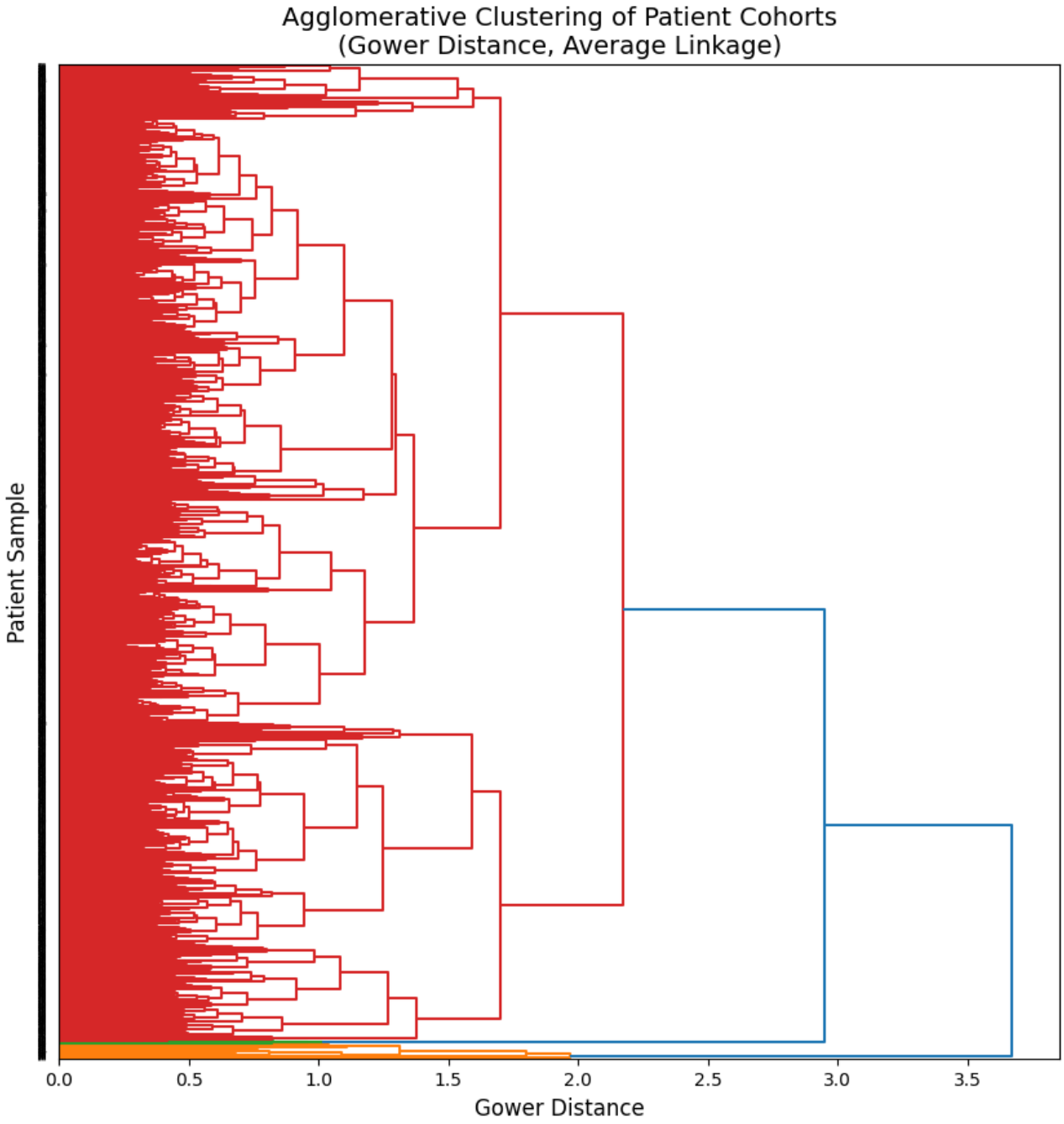


Figure 20: Dendrogram from agglomerative hierarchical clustering of 3,000 lung cancer patients using average linkage and Gower distance. A horizontal cut at distance 2.1 yields four distinct patient clusters.

The dendogram implies there are three different patient groups within the given patient subset (from the three different colours). As there is one dominant group, the dendogram supports the assumption made following from the boxplots within the exploratory data analysis section. The dendogram was used to provide guidance for the distance threshold (evaluated to be 2.1) for grouping patients into a cluster, this led to the creation of four clusters upon calling the scipy.fcluster method, which is a method that creates cluster assignments through cutting the linkage clustering tree based on the distance threshold.

Cluster ID	Cluster Size
1	47
2	6
3	961
4	1986

Table 11: Table showing the size of each cluster.

Since the size of cluster 2 is less than 10, it is statistically insignificant and is thus removed.

11.1 Evaluating the Cluster

To visualise the differences between each cluster, a heatmap normalised using min max normalisation was implemented. Min max normalisation was used as opposed to other forms of normalisation in order to get a bounded output between 0 and 1.



Figure 21: The first half of the normalised heatmap, showing the mean value for each feature across clusters. Green indicates higher relative values and red indicates lower relative values.

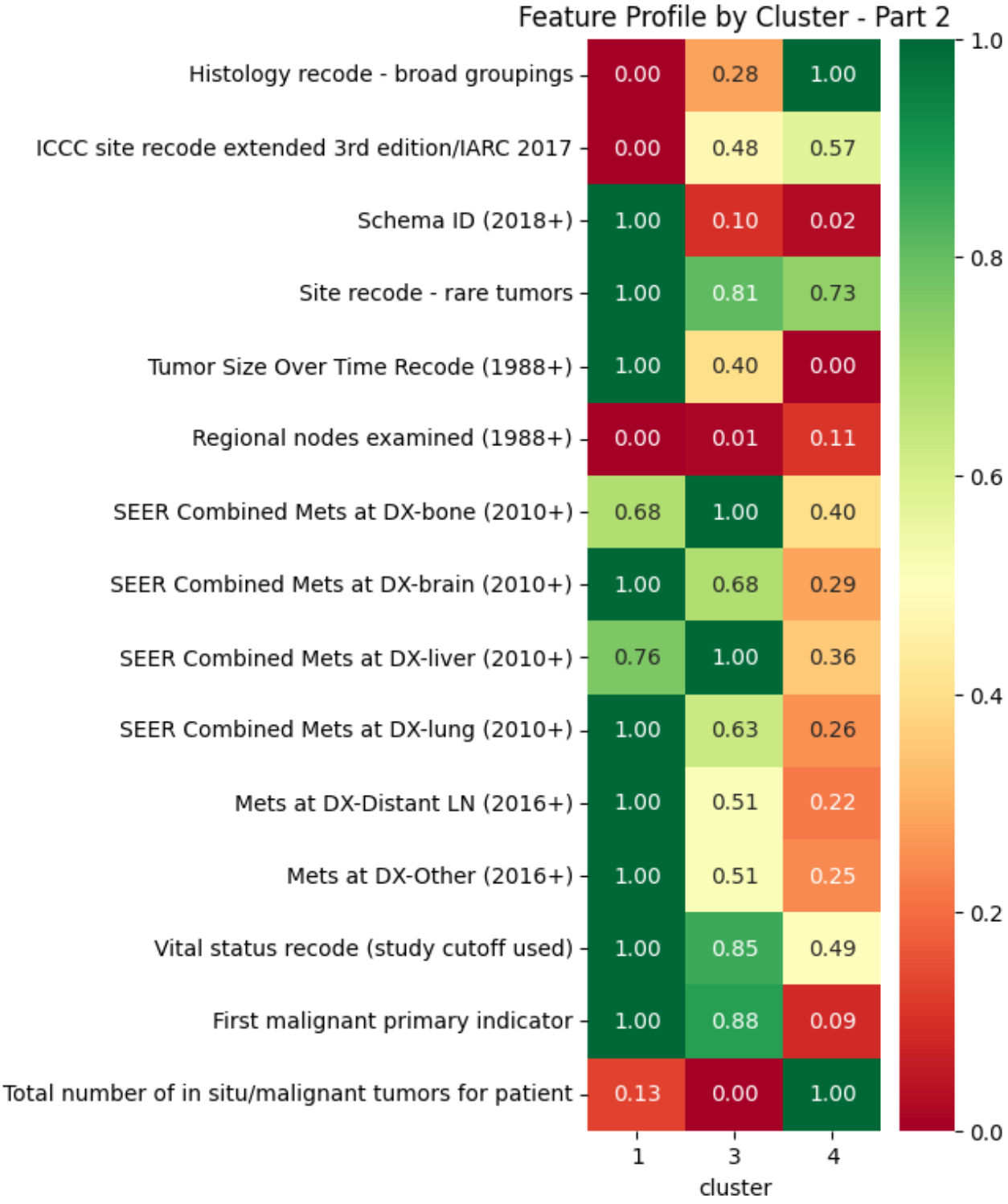


Figure 22: The second half of the normalised heatmap.

Out of the three clusters, the top ten covariates with the highest variance across clusters were chosen, as shown in the table below.

Table 12: Top 10 Covariates by Variance Across Clusters

Covariate	**Variance**
Total number of in situ/malignant tumors for patient	0.296
Race recode (W, B, AI, API)	0.285
Age recode with <1 year olds and 90+	0.275
Sex	0.274
Histology recode - broad groupings	0.266
Tumor Size Over Time Recode (1988+)	0.253
First malignant primary indicator	0.247
Primary Site - labeled	0.203
Mets at DX-Distant LN (2016+)	0.155
Mets at DX-Other (2016+)	0.146

To rank the clusters shown in the heatmaps by their average survival probability belonging to the patients within these clusters, in order to identify which clusters represented patients with higher survival risk, with the aim of determining associations between covariates and risk, the Kaplan Meier model was used. Below is a table showing the normalised values for the top ten covariates with the highest variance along with an extra column providing the corresponding survival probabilities for each cluster as given by fitting the survival months and event statuses of each patient in a cluster into the Kaplan Meier (KM) model, and getting the models output probability for surviving 60 months. The tables are plotted below along with the "Survival Probability at 60 Months" column for easier identification of the covariates that have a direct relationship with the survival probability (as the value of the covariate increases / decreases the corresponding survival probability should show a trend). Other covariates that do not appear to follow an association with survival probability include "SEER Combined Mets at DX-liver" and "Total number of in situ/malignant tumours for patient".

Table 13: Normalized Feature Profiles by Cluster with 60-Month Survival Probability

Cluster	**Survival Probability at 60 Months**	**Tumors**	**Race**	**Age**	**Sex**	**Histology**
1	0	0.130	0.000	0.000	1.000	0.000
3	0.151	0.000	0.822	1.000	0.231	0.284
4	0.424	1.000	1.000	0.771	0.000	1.000

Table 14: Normalized Feature Profiles by Cluster with 60-Month Survival Probability

Cluster	Survival Probability at 60 Months	Size	Primary	Site	Mets LN	Mets Other
1	0.0000	1.000	1.000	0.140	1.000	1.000
3	0.1513	0.402	0.882	1.000	0.514	0.515
4	0.4237	0.000	0.086	0.801	0.222	0.245

Cluster 4 has the highest survival probability, even though the number of tumours is highest. There is an association between the survival probabilities derived by the KM model of a patient and their gender. Cluster 4 is derived of majority female patients, and has the highest survival probability, whilst cluster 1 is derived of majority male patients, having the lowest survival probability. A potential reason to why the 'number of tumours' do not show a relationship between the survival probabilities could be due to the impact of the number of tumours being dependent on the location of the tumours within the lung (represented by the "Primary Site - labeled" covariate). This hypothesis is supported in [14] which is a cancer registration study showing that patients with lower lobe tumours had poorer survival compared to patients with tumours in the upper and middle lobes.

Other covariates noted to not follow an association with survival probability include SEER Combined Mets at DX-bone (2010+) and SEER Combined Mets at DX-liver (2010+). SEER Combined Mets at DX-bone (2010+) represent whether the cancer has spread from the primary site (lung). It would be reasonable to investigate for hidden relationships between the number of tumours and whether there has been metastasis in any other regions, this follows from how the impact of tumour size depends on location, and tumours that have spread beyond the primary region typically have a much stronger impact than a high number of tumours that have stayed within the region [20].

Table 14 also shows an association between median household income and survival probability. The scatterplots are illustrated below to further investigate this association.

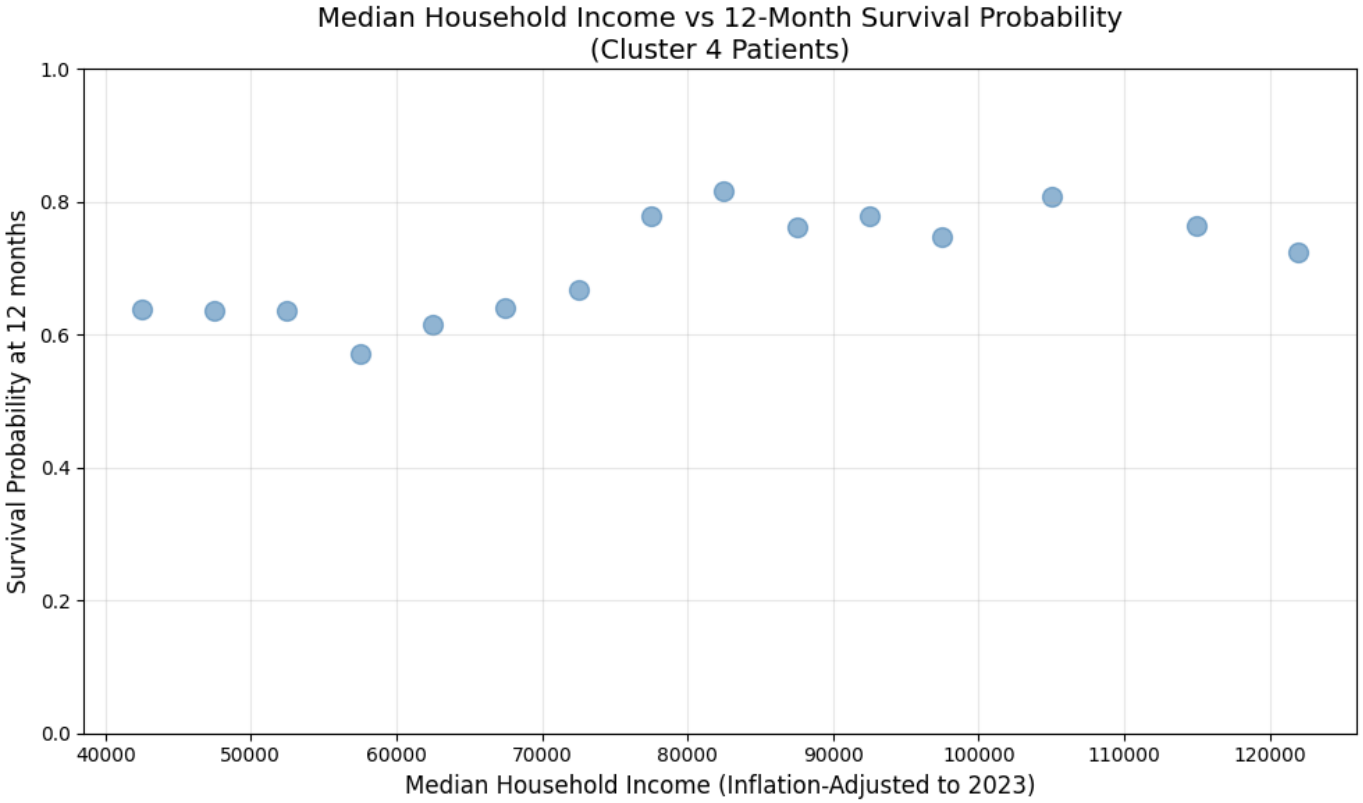


Figure 23: The scatterplot showing the linear relationship between the household income (represented on the x axis as the midpoint of an income bin) and the median survival probabilities of the patients within cluster 4 that fall in the particular income bin.

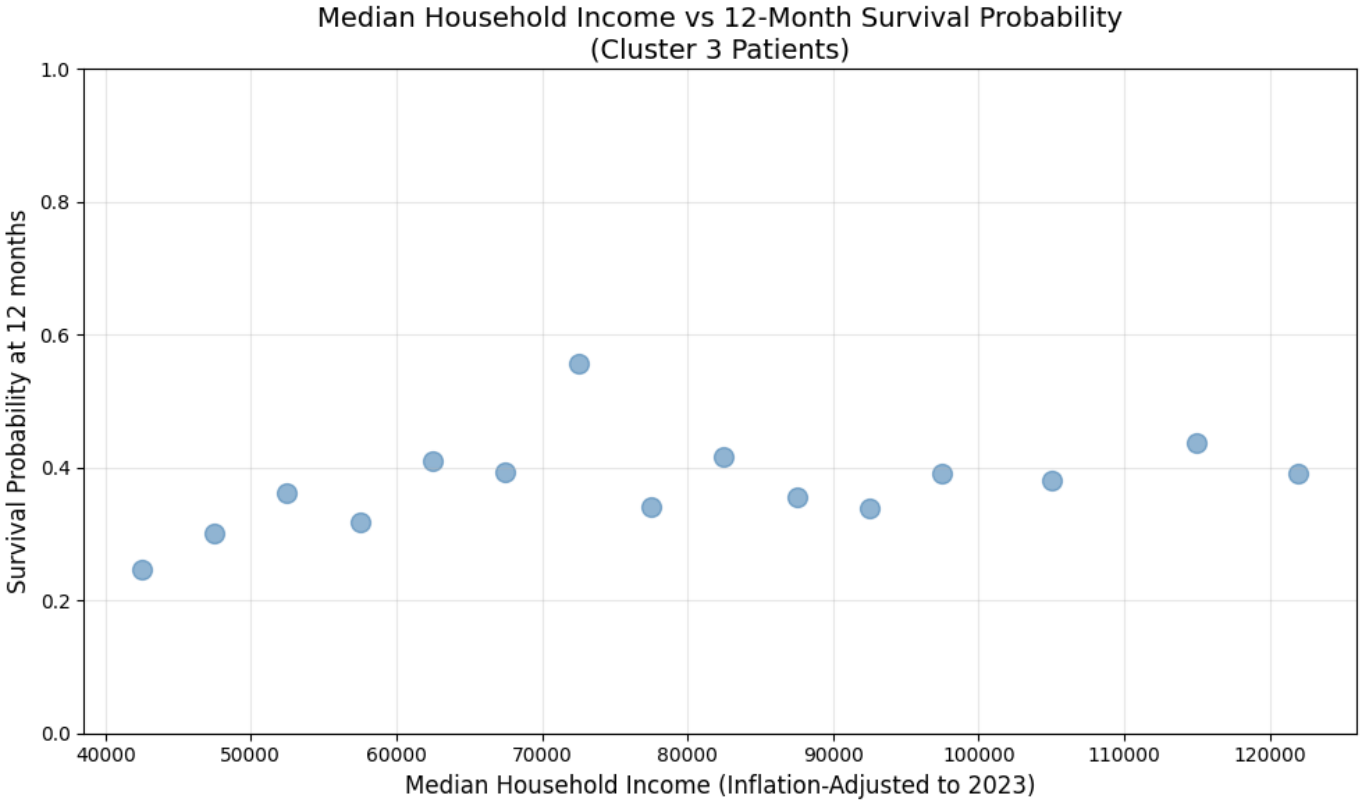


Figure 24: The scatterplot showing the relationship as above but for patients in cluster 3.

Both scatter plots show a positive trend between median household income and survival probability determined by the Kaplan Meier model.

12 Wald Test to Evaluate New Features

Following from findings from the previous section, three feature engineered covariates were created to investigate the hidden relationships between the covariates that did not follow an association with survival probability: Tumour Number Bone Adjusted; Tumour Number Liver Adjusted; Tumour Number Site Adjusted, which are defined as follows:

$$\text{Tumour Number Adjusted} = \begin{cases} \text{Number of Tumours}, & \text{if Primary Site Labelled} = \text{`C34.1 – Upper lobe, lung'} \\ \text{Number of Tumours} \times 4, & \text{if Primary Site Labelled} = \text{`C34.2 – Middle lobe, lung'} \\ \text{Number of Tumours} \times 2, & \text{if Primary Site Labelled} = \text{`C34.3 – Upper lobe, lung'} \\ \text{Number of Tumours} \times 4, & \text{if Primary Site Labelled} = \text{`C34.9 – Lung, NOS'} \\ \text{Number of Tumours} \times 5, & \text{if Primary Site Labelled} = \text{`C34.0 – Main bronchus'} \\ \text{Number of Tumours} \times 6, & \text{if Primary Site Labelled} = \text{`C34.8 – Overlapping lesion of lung'} \end{cases}$$

Where Tumor Number Adjusted is the newly defined variable derived from both the number of tumours and the site of the tumours. The factors based on the location were determined by ranking survival chances based on location.

$$\text{Tumour Number Liver} = \begin{cases} \text{Number of Tumours} \times 2, & \text{if SEER Combined Mets at DX-liver (2010+)} = \text{Yes} \\ \text{Number of Tumours}, & \text{if SEER Combined Mets at DX-liver (2010+)} = \text{No} \end{cases}$$

Where Tumour Number Liver is the tumour number multiplied by 1.5 if liver metastasis has occurred, as liver metastasis would lead to a worse prognosis.

$$\text{Tumour Number Bone} = \begin{cases} \text{Number of Tumours} \times 1.5, & \text{if SEER Combined Mets at DX-bone (2010+)} = \text{Yes} \\ \text{Number of Tumours}, & \text{if SEER Combined Mets at DX-bone (2010+)} = \text{No} \end{cases}$$

Where Tumour Number Bone Adjusted is the tumour number multiplied by 1.5 if Bone metastasis has occurred, as bone metastasis would lead to a worse prognosis.

To ensure that the engineered covariates (if any) are statistically significant in order to avoid using time unproductively on optimising an XGBoost model with insignificant features, the Wald test at the 5% significance level will be performed separately for each of the three covariates using the Cox Proportional Hazards (CoxPH) model. The CoxPH model was chosen since it is a regression model used for survival analysis, specific to the dataset by taking into account both the time and event for its output. The Wald test was the chosen testing method (out of the Likelihood Ratio Test and Logrank test) as its main use is to test individual coefficients cite. The Wald test is calculated using lifelines CoxPHFitter().fit() and displayed as one of the metrics from CoxPHFitter().summary().

To perform the Wald test for the Tumour Number Bone covariate, the null model was:

$\eta(x) = \beta_1$Mets at DX Bone $+ \beta_2$Number of Tumours

and the alternate model being:

$\eta(x) = \beta_1$Mets at DX Bone$+\beta_2$Number of Tumours$+\beta_3$Tumour Number Bone

To perform the Wald test for the Tumour Number Liver covariate, the null model was:

$\eta(x) = \beta_1$Mets at DX Liver $+ \beta_2$Number of Tumours

and the alternate model being:

$\eta(x) = \beta_1$Mets at DX Liver$+\beta_2$Number of Tumours$+\beta_3$Tumour Number Liver

To perform the Wald test for the Tumour Number Adjusted covariate, the null model was:

$\eta(x) = \beta_1$Primary Site Labelled $+ \beta_2$Number of Tumours

and the alternate model being:

$\eta(x) = \beta_1$Primary Site Labelled$+\beta_2$Number of Tumours$+\beta_3$Tumour Number Adjusted

The p values for the coefficients representing Tumour Number Bone, Tumour Number Liver and Tumour Number Adjusted was 0.0238, 0.920, 0.0213 (all to three significant figures) respectively. At the 5% significance level, the null hypotheses for "Tumour Number Bone Adjusted" and "Tumour Number Site Adjusted" was rejected. However, the alternate hypothesis for Tumour Number Liver Adjusted was rejected.

Note that Tumour Number Liver has a higher scaling factor than "Tumour Number Bone Adjusted" yet its alternate hypothesis was rejected. Reasons could be due to the nature of the dataset. Potentially the association between "Mets DX Liver" and the survival duration was stronger than that of "Mets DX Bone" and the survival duration, meaning the engineered variable had a weaker impact on model output, and that the covariate "Mets DX Bone" does not have a significant impact on the patients survival probability, however, research contradicts this, stating that bone metastasis does significantly increase patient risk.

Determined by the Wald test results, the engineered covariates Tumour Number Bone and "Tumour Number Adjusted" will be added to the feature matrix during the XGBoost training, optimising and testing process.

13 Impact of Feature Engineered Covariates on the XGBoost Model

After adding the two feature engineered covariates to the feature matrix and re-iterating through the optimisation and testing process. The first and second optimised configurations attained a KM score of 0.393 and 0.307 (3sf) respectively.

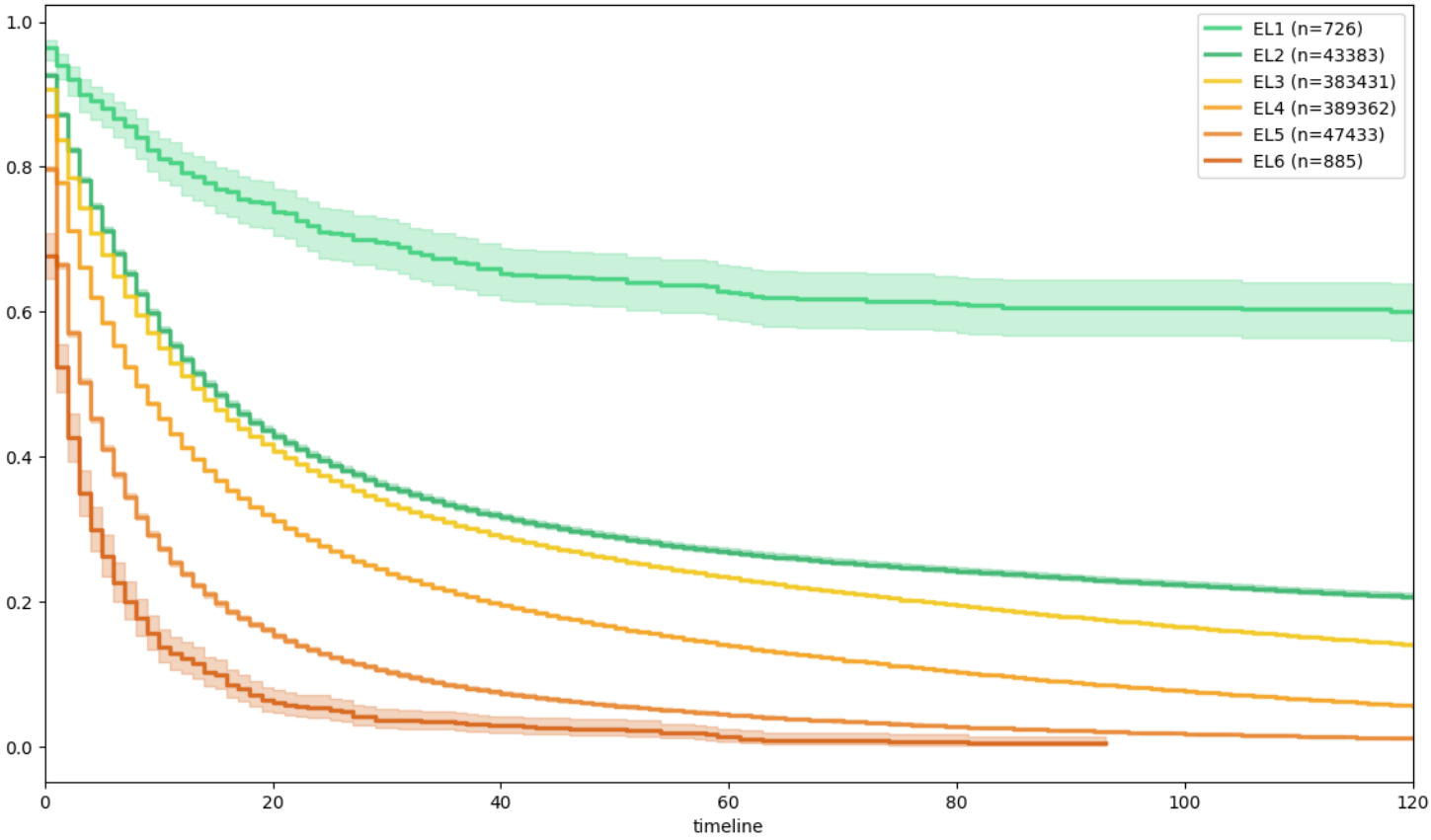


Figure 25: KM plot for the first optimised configuration.

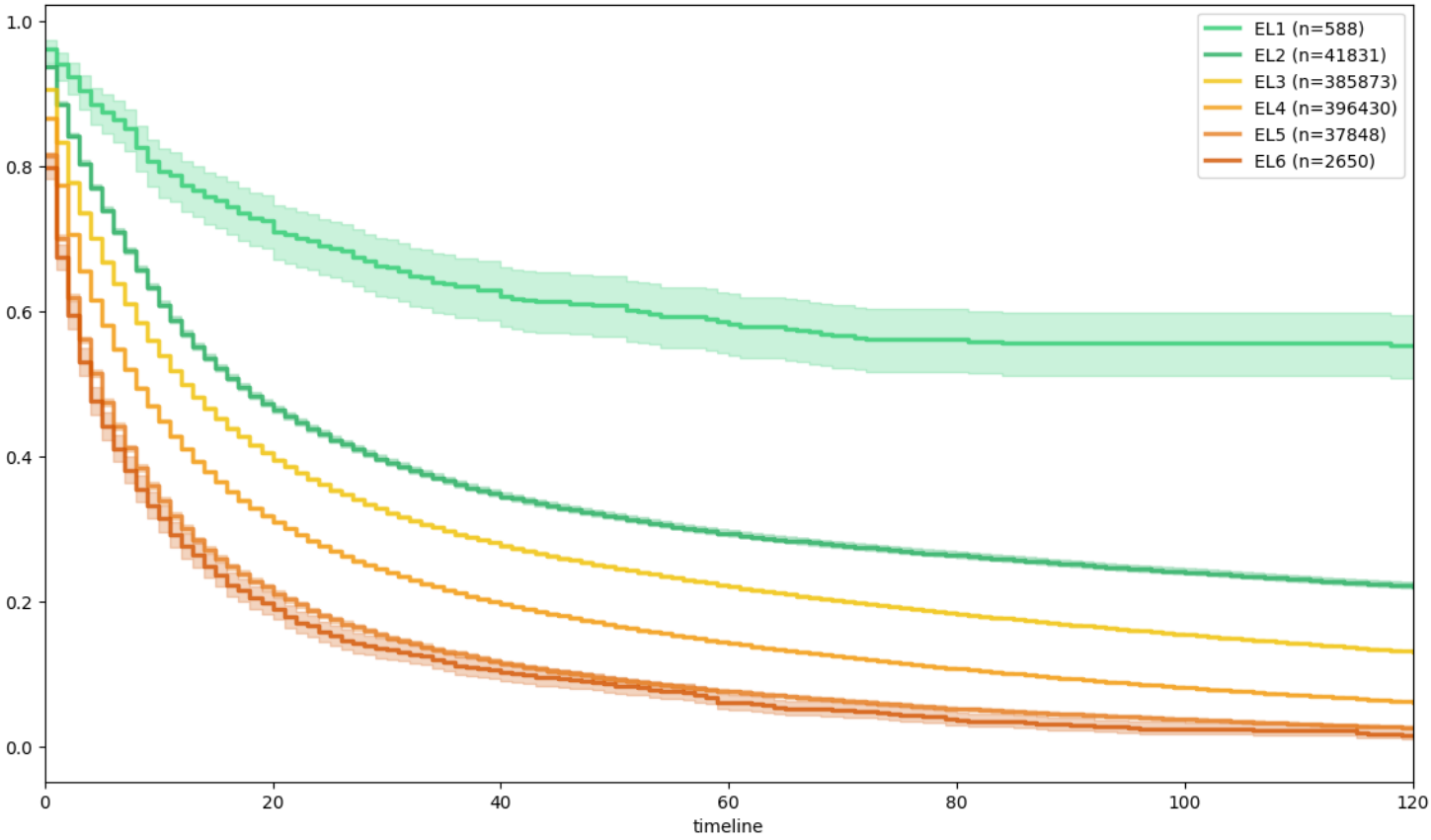


Figure 26: KM plot for the second optimised configuration

The KM plot for the first configuration did not show improvement from the KM plots generated by the XGBoost model when trained using the original feature matrix. However, the second configuration demonstrated significant improvement. The curves representing EL2 to EL4 displayed traits for a perfect KM plot that would have a KM score of 0, however, uniform separation between all curves was still not achieved.

14 Evaluation

Out of the three models in the project, the Random Survival Model had the lowest (best) KM score, producing a KM plot closest to one whose curves are of equal separation out of the other two models (DeepHit and XGBoost). There is insufficient evidence to claim that it is possible to create a KM plot where consecutive curves follow equal separation. The boxplots shown during EDA imply that there exists covariates with many outliers, supporting the idea that

curves of perfectly equal separation are unattainable due to many patients not belonging to a particular phenotype. The presence of many outliers suggests patients don't conform cleanly to distinct phenotypes, producing heterogeneous survival trajectories within each group and preventing uniform separation between curves. Potentially, the relationships between covariates and the survival duration and event status are not strong enough for the model to detect in such a way that patients can be classified into risk labels that form KM plots of equal separation (as implied by the optimal configurations for the XGBoost model), due to similar values for covariates leading to significantly different survival durations, as shown by the scatter plots investigating the top six covariates ranked by permutation importance for the DeepHit model, whereby for five of the six scatter plots, no association between the covariates and the survival duration could be visually detected. The table below re states the objective functions and their justifications on how they have been satisfied.

Objective	**Justification of Accomplishment**
Investigate the efficacy of a shallow learning model as a classification method for a labelling system that leads to a KM plot whose survival curves are of uniform separation over time.	A Random Survival Forest model was implemented, whereby after optimisation with the KM score as the objective function, the model failed to produce KM plots whose survival curves are of uniform separation over time.
Investigate the efficacy of a deep learning model as a classification method for a labelling system that leads to a KM plot whose survival curves are of uniform separation over time.	A DeepHit model was implemented, whereby after optimisation with the KM score as the objective function, the model failed to produce valid KM plots, due to survival curves intersecting, which imply that risk groups are not distinct or ordered.

Table 15: Table showing the original objectives and their justifications of accomplishment.

Note that from a visual stand point, the Random Survival Forest model led to better KM plots than the Tumour Node Metastisis (TNM) staging system, as the surival curves provided more risk profiles than just four survival curves and covered the survival probability region above 0.4 beyond the ten year time point, unlike the TNM staging system.

14.1 Reflection

More time could have been allocated from developing solutions to further investigating the results. This could have led to better improvements in the KM score for models. In addition, further exploratory data analysis could have been performed at the start of the project in order to develop more data specific solutions for generating better KM plots.

Difficulties encountered included designing and implementing the scoring function, as well as creating the evaluation function to be passed to the Bayesian optimiser. Another difficulty faced was in determining methods and approaches in response to results (due to ambiguitity involving the next best approach), such as determining what to do next after attaining the KM score corresponding to the DeepHit model, as the objectives were already satisfied by then. Determining responses to unexpected results was also challenging, such as determining alternate methods to evaluate the permutation importance for the Random Survival Forest model when scikit's permutation_importance function was unable to be used due to its high runtime.

15 Further Extensions

Below lists further extensions for this project.

- Implement transformer tabular diffusion to generate synthetic data with the aims of improving representation within under represented patient risks groups, with the aim of generating models whose predictions lead to survival curves of equal separation.
- Compare the Random Survival Forest and DeepHit staging systems against established clinical prognostic tools.
- Investigate alternative labelling strategies such as quantile based binning.
- Investigate the optimal number of curves with regards to minimising the KM score.
- Investigate the impact of imputation on improving model performance.

References

[1] Amazon Web Services. Tune an xgboost model (amazon sagemaker documentation). `https://docs.aws.amazon.com/sagemaker/latest/dg/xgboost-tuning.html`, 2026. Accessed: 2026-04-04.

[2] Norman E. Breslow. Discussion of the paper by D. R. Cox. *Journal of the Royal Statistical Society: Series B (Methodological)*, 34(2):216–217, 1972.

[3] Hanqing Cao, Larry Eshelman, Nicolas Chbat, Larry Nielsen, Brian Gross, and Mohammed Saeed. Predicting icu hemodynamic instability using continuous multiparameter trends. In *2008 30th Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, pages 3803–3806, 2008.

[4] Mary A. Duggan, Warren F. Anderson, Sean Altekruse, Lynne Penberthy, and Margaret E. Sherman. The surveillance, epidemiology, and end results (SEER) program and pathology: Toward strengthening the critical relationship. *The American Journal of Surgical Pathology*, 2016.

[5] Amin et al. Ajcc cancer staging manual. *CA: A Cancer Journal for Clinicians*, 2017.

[6] Choi et al. Using recurrent neural network models for early detection of heart failure onset. *Journal of the American Medical Informatics Association*, 2017.

[7] Churpek et al. Predicting clinical deterioration in the hospital: The impact of outcome selection resuscitation. *Resuscitation*, 2013.

[8] Futoma et al. Learning to detect sepsis with a multitask gaussian process rnn classifier, 2017. https://arxiv.org/abs/1706.04152.

[9] Hendriksen et al. Diagnostic and prognostic prediction models. *Journal of Thrombosis and Haemostasis*, 2013.

[10] Ishwaran et al. Random survival forests. *The Annals of Applied Statistics*, 2008.

[11] Jarvis et al. Development and validation of a decision tree early warning score based on routine laboratory test results for the discrimination of hospital mortality in emergency medical admissions. *Resuscitation*, 2013.

[12] Lee et al. Dynamic-deephit: A deep learning approach for dynamic survival analysis with competing risks based on longitudinal data. *IEEE*, 2020.

[13] Shamout et al. Machine learning for clinical outcome prediction. *IEEE*, 2021.

[14] Wang et al. Lung cancer and prognosis in taiwan: a population-based cancer registry. *Journal of Thoracic Oncology*, 8(9):1128–1135, 2013.

[15] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2 edition, 2009.

[16] Yuan He, Haiyan Li, Lei Wang, Qian Zhang, You Zheng, Fang Li, Bin Liu, Long Chen, Jie Zeng, Yuan Zhang, Le Zhao, Qian Wang, Haifeng Guo, Jie Wang, and Liyuan Ma. Pan-cancer analysis reveals sex-specific differences in the association between immune-related gene expression and patient survival. *MedComm*, 3(3):e144, 2022. PMCID: PMC9246337.

[17] Changhee Lee, William R. Zame, Jinsung Yoon, and Mihaela van der Schaar. Deephit: A deep learning approach to survival analysis with competing risks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[18] J.-G. Lee, C.-Y. Lee, I.-K. Park, D.-J. Kim, and K.-Y. Chung. The number of metastatic lymph nodes as a prognostic factor in patients with N1 non-small cell lung cancer. *The Annals of Thoracic Surgery*, 86(4):1177–1183, 2008.

[19] Ethan Lim. Interview with prof eric lim, September 2025. Personal interview conducted by the author.

[20] National Cancer Institute. Seer cancer statistics review (csr) 1975–2017: Lung and bronchus – 5-year relative survival by stage at diagnosis (2010–2016). `https://seer.cancer.gov/csr/1975_2017/browse_csr.php?pageSEL=sect_15_table.13&sectionSEL=15`. Accessed: 2026-04-09.

[21] National Cancer Institute. Cancer staging, 2022. https://www.cancer.gov/about-cancer/diagnosis-staging/staging.

[22] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 2019.

[23] scikit-learn developers. Decision trees — scikit-learn documentation. `https://scikit-learn.org/stable/modules/tree.html`, 2026. Accessed: 2026-04-07.

[24] scikit-learn developers. Random forests and other randomized tree ensembles — scikit-learn documentation. `https://scikit-learn.org/stable/modules/ensemble.html#random-forests`, 2026. Accessed: 2026-04-07.

[25] scikit-optimize developers. skopt.gp_minimize — scikit-optimize documentation. `https://scikit-optimize.github.io/stable/modules/generated/skopt.gp_minimize.html`, 2026. Accessed: 2026-04-07.

[26] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.

[27] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical Bayesian optimization of machine learning algorithms. *Advances in Neural Information Processing Systems*, 2012.

[28] Ailin Yan, Jinqian Lv, Yukun Yuan, Huaping Yang, Fu Li, Haiyan Yu, Shun Zhao, Guowei Zhang, and Guorong Zhang. Primary site and its effect on survival in patients with adenocarcinoma and squamous cell carcinoma of the lung. *Chinese Medical Journal*, 135(2):127–136, 2022. PMCID: PMC8769119.

A Appendix: Additional Material

The listing below provides the code behind the functions used to create Kaplan Meier (KM) plots and the corresponding KM score, given a dataset containing the risk labels assigned to each patient as well as their survival duration and event status.

Listing 1: Code for the Kaplan-Meier (KM) plotter as well as the KM scoring function

```

from lifelines import KaplanMeierFitter
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
import itertools

class KM_Plotter():

    def __init__(self,risk_labels_df,num_curves=10):
        self.risk_labels_df = risk_labels_df
        self.num_curves = num_curves
        self.survival_functions = []
        self.survival_functions_temp = []
        self.optimal_separation = -1 #to be initialised

    def makeKMPlot(self,plot=True):
        km_data = pd.DataFrame({
            'time': self.risk_labels_df['Survival_Time'],
            'event': self.risk_labels_df['Event_Status'].astype(int), #0 -
                 censored
            'risk_label': self.risk_labels_df['Risk_Label']
        })
        if plot:
            plt.figure(figsize=(14,8))
        kmf = KaplanMeierFitter()

        #Define colours for each risk group
        color = ['#2ecc71', '#27ae60', '#f1c40f', '#f39c12', '#e67e22',
               '#d35400', "#a55e56", '#c0392b', '#8e44ad', '#2c3e50']

        risk_labels_sorted = [f'EL{i}' for i in range(1,self.num_curves
             +1)]

        for i,label in enumerate(risk_labels_sorted):

            mask = km_data['risk_label'] == label
            n_patients = mask.sum()

            if n_patients > 0:
```

```
            kmf.fit(
                durations = km_data.loc[mask,'time'],
                event_observed = km_data.loc[mask,'event'],
                label = f'{label} (n={n_patients})'
                )

            self.survival_functions.append(kmf.survival_function_)

            print(f"appending curve for risk label {i}")
            if plot:
                ax = kmf.plot_survival_function(
                    color=color[i],
                    linewidth=2.5,
                    alpha=0.8 #changing increases length of confidence region but decreases size of box
                )
                plt.xlim(0, 120)

        self.survival_functions_temp = self.survival_functions

        #take each survival function in our survival functions list and join them accordingly
        # Create timeline from 0 to 150 with step size 1
        timeline = pd.DataFrame({'timeline': np.arange(0, 151, 1)})

        #temp is a list of DataFrames (one per risk group), each with time as index and survival probability as value
        #we want to left join each survival function onto the timeline by the index (time)
        merged = timeline.copy()
        print("Survival functions df is: ")
        print(self.survival_functions)
        print("Length of temp survival functions is: ", len(self.survival_functions_temp))
        print(f"Length of survival functions is: {len(self.survival_functions)}")
        for i, survival_function in enumerate(self.survival_functions):
            print(f"{i}th enumeration")
            # Reset index to get time as a column, rename survival prob column
            surv_df = survival_function.reset_index()
            surv_df.columns = ['timeline', f'{i+1}']
            merged = merged.merge(surv_df, on='timeline', how='left')
        #cut off timeline values beyond 120
        merged = merged[merged['timeline'] <= 120].reset_index(drop=True)
        self.survival_functions = merged
        self.optimal_separation = 1/self.num_curves
        print(f"length of survival functions is: {len(self.survival_functions)}")

```

```
81      def getKMScore(self, missing_penalty=1):
82          #penalise incomplete curves
83          #score baseed off of distance between curves relative to 1/(
                number of curves)
84          #get all pairwise combinations of columns
85          #labels should have order and there should be no overlap
86          print("Update 16")
87          print(f"Length of survival functions: {len(self.
                survival_functions)}")
88          print(f"survival functions columns: {self.survival_functions.
                columns}")
89          #from 0 to num_curves-1, get consecutive pairs of columns
90          consecutive_pairs = [(i, i+1) for i in range(1,len(self.
                survival_functions.columns)-1)]
91          parallelism_scores_df = pd.DataFrame()
92          pair_scores = []
93          missing_curves = self.num_curves - len(self.survival_functions.
                columns) + 1
94          if missing_curves > 0: #don't want missing curves
95              return 1000
96
97          #ensure ranking between risk labels is conserved and there is no
                overlap
98          for col_index in range(1, len(self.survival_functions.columns)-1)
                :
99              for row in range(len(self.survival_functions)):
100                 #if value for row and col_index is higher than the same
                        row for all proceeding columns, penalise
101                 value = self.survival_functions.iloc[row, col_index]
102                 if not pd.isna(value) and not np.isinf(value):
103                     for proceeding_col_index in range(col_index+1, len(self
                            .survival_functions.columns)-1):
104                         proceeding_value = self.survival_functions.iloc[row
                                , proceeding_col_index]
105                         if not pd.isna(proceeding_value) and not np.isinf(
                                proceeding_value):
106                             if value <= proceeding_value:
107                                 return 1000 #worst possible score
108
109         for consecutive_pair in consecutive_pairs:
110             pair_differences = []
111             print(f"Consecutive pair being compared is: {consecutive_pair}
                    ")
112             for row in range(len(self.survival_functions)):
113
114                 value_1 = self.survival_functions.iloc[row,
                        consecutive_pair[0]]
115                 value_2 = self.survival_functions.iloc[row,
                        consecutive_pair[1]] # higher value
116                 if not pd.isna(value_1) and not np.isinf(value_1) and not
```

```
                    pd.isna(value_2) and not np.isinf(value_2):
117                 difference = abs((value_2 - value_1 - self.
                        optimal_separation))
118             else:
119                 difference = missing_penalty
120             print(f"difference: {difference}")
121             pair_differences.append(difference)
122
123         if len(pair_differences) == 0:
124             average_difference = missing_penalty
125         else:
126             average_difference = np.mean(pair_differences)
127
128         parallelism_scores_df[str(consecutive_pair)] = [
                average_difference]
129         pair_scores.append(average_difference)
130
131
132     if parallelism_scores_df.empty or len(pair_scores) == 0:
133         print("scores are empty, returning 1000")
134         return 1000 #worst possible predictions
135     else:
136         average = np.mean(pair_scores)
137         if np.isnan(average):
138             print("average is NaN, returning worst score of 1000")
139             return 1000
140         else:
141
142             return average
```